

Technical paper

Data-driven hierarchical multi-policy deep reinforcement learning framework for multi-objective multiplicity dynamic flexible job shop scheduling

Linshan Ding^a, Zailin Guan^b, Dan Luo^b, Lei Yue^{a,*}

^a School of Mechanical and Electrical Engineering, Guangzhou University, Guangzhou, PR China

^b Department School of Mechanical Science and Engineering, Huazhong University of Science and Technology, Wuhan, PR China

ARTICLE INFO

Keywords:

Hierarchical multi-policy deep reinforcement learning
Data-driven
Multiplicity flexible job shop scheduling
Dynamic scheduling
Multi-objective optimization

ABSTRACT

In the context of Industry 4.0, manufacturers face pressure to personalize products and accelerate the supply chain. This requires rapid response to volatile production schedules, ensuring a balance between operational efficiency and product quality. Moreover, the rapid development and convergence of the cloud computing, Internet of Things (IoT), and big data have expanded the need for real-time tracking and adaptive scheduling to address uncertainties, such as equipment downtime, supply variation, and ongoing product revisions. The capability of IoT has significantly improved the continuous monitoring and data analysis, emphasizing the importance of developing effective real-time scheduling solutions in the manufacturing system. In response to these evolving industrial requirements, and driven by objectives to reduce the makespan, total tardiness, and energy consumption, we study the multi-objective multiplicity dynamic flexible job shop scheduling problem (MOMDFJSP), to cope with the challenges of new order arrivals and machine breakdowns in the IoT-enabled manufacturing system. This study proposes a novel hierarchical multi-policy deep reinforcement learning framework for IoT-infused manufacturing environments, aiming to integrate these diverse requirements and uncertainties into a coherent and responsive scheduling framework. The proposed framework comprises an upper-level control policy network and three lower-level objective policy networks. The upper-level and lower-level networks are respectively responsible for selecting temporary optimization objectives and specific dispatching rules. Based on the proposed framework, we design a two-stage training approach named the hierarchical multi-policy soft actor-critic (HMPSAC) algorithm to train multiple policy networks. In addition, we develop a fluid model to design the state features and dispatching rules that act as inputs and outputs, respectively, for the deep reinforcement learning (DRL) policy network. The comparative analysis with well-known dispatching rules and DRL-based methods reveals the superior performance of HMPSAC algorithm.

1. Introduction

With the acceleration of economic globalization and the intensification of market competition, notable changes are emerging in the manufacturing industry. The manufacturing paradigm has shifted from a manual, extensive, and passive mode to a transparent, lean, and proactive approach. In modern manufacturing systems, the dynamic interplay of machines, tools, materials, and labor leads to fluctuant scenarios. Emergent issues such as machine breakdowns, tool degradation, material inconsistencies, and labor fluctuations, compounded by unforeseen orders, underscore the need for resilient scheduling

frameworks[1]. These frameworks must dynamically adapt to such variability. The inherent dynamics of manufacturing environments drive research into the dynamic flexible job shop scheduling problem (DFJSP), where the goal is to optimize the allocation of jobs to resources while adeptly navigating the complexities introduced by such dynamic events.

As a classical production scheduling problem, the DFJSP involves random job arrivals, uncertain processing times, and alterations in due dates. Then, DFJSP is a more complex NP-hard problem than static flexible job shop scheduling problem (FJSP) and job shop scheduling problems (JSP)[2]. In numerous manufacturing environments, the prevailing operational mode is characterized by multi-variety and

* Correspondence to: School of Mechanical and Electrical Engineering, Guangzhou University, Higher Education Mega Center, 230 Wai Huan Xi Road, Guangzhou 510006, PR China.

E-mail addresses: linshan_ding@gzhu.edu.cn (L. Ding), zlguan@hust.edu.cn (Z. Guan), luodan231204@foxmail.com (D. Luo), leileiyok@gzhu.edu.cn (L. Yue).

<https://doi.org/10.1016/j.jmsy.2025.03.019>

Received 17 March 2024; Received in revised form 15 January 2025; Accepted 21 March 2025

Available online 4 April 2025

0278-6125/© 2025 The Society of Manufacturing Engineers. Published by Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

small-batch production. This approach typically entails the production of a diverse array of products in relatively modest quantities, necessitating a high degree of flexibility and efficiency in the scheduling processes [3]. In this paper, instead of considering the division of batches, the multi-variety and small-batch production is represented as multiplicity scheduling in which each job in the batches of different job types is considered as an individual job and is processed separately. Although this approach increases the scale of the problem, it creates a customized schedule for every job, potentially uncovering deeper layers of optimization potential. Furthermore, with the increasingly severe environmental challenges and growing emphasis on energy conservation and emissions reduction, reducing workshop energy consumption has become more important than ever. Therefore, we explore the multi-objective multiplicity dynamic flexible job shop scheduling problem, considering three objectives: total energy consumption, total tardiness, and maximum completion time, as well as two types of dynamic events: machine breakdowns and the arrival of new orders.

The MOMDFJSP is a variant of the standard DFJSP where each job type has multiple jobs to be processed. MOMDFJSPs can be highly beneficial in real-life production scenarios such as semiconductor manufacturing systems and motorcycle part manufacturing lines [4]. Addressing the MOMDFJSP can help companies improve throughput, reduce lead times, minimize the idle time of machines, and enhance resource utilization. Companies can achieve higher productivity and responsiveness to customer demands by effectively managing the scheduling of multiple job instances, resulting in improved competitiveness in the market. Given its high complexity and broad applicability, research on the MOMDFJSP is essential for academia and industry.

In addition, the rapid development of advanced information technologies, including cloud computing, the IoT, and cyber-physical systems (CPS), led to the conceptualization of the IoT-based manufacturing system (IoTMS). The IoTMS enables advanced management measures, such as timely data collection, rapid response to production disturbances, and accurate execution of production decisions [5,6]. Therefore, recent literature on real-time online scheduling in manufacturing systems emphasizes the integration of IoT and DRL for dynamic adaptability [7]. However, current DRL-based scheduling multi-objective optimization methods encounter two main challenges. Firstly, existing DRL-based methods primarily use single-stage training method which trains multi-policy networks simultaneously, leading to issues like distribution shift, coordination problems, and a lack of diversity in Pareto front solutions. This method overlooks crucial factors such as variable objective weights and Pareto front uniformity. Secondly, most DRL-based methods derive state features from the problem's characteristics and select classical dispatching rules as action sets, lacking exploration of effective state representations and the short-sightedness of action rules.

To address these challenges, the data-driven hierarchical multi-policy deep reinforcement learning framework and two-stage training method are proposed. In the proposed method, we leverage the advantages of fluid models in continuous system modeling to approximate the production process as a continuous flow of fluid, thereby assisting in the extraction of state features. Moreover, we utilize the asymptotic optimality of fluid models to design scheduling rules, aiming to improve DRL's decision-making performance and global optimization capabilities. This method tracks dynamic events in real-time for immediate resolution of fluid models, merging these dynamic parameters with optimization objectives to customize state features and scheduling rules. These state features and scheduling rules are not only closely related to the discrete processing state of the workshop but also linked to the dynamic parameters that evolve in the fluid model. This strategy significantly enhances the recognition and decision-making capabilities of DRL in workshop scheduling. The main contributions of this study include:

- (1) Different from the traditional DRL frameworks for scheduling, the proposed data-driven hierarchical multi-policy deep reinforcement learning framework develops separate policy networks for different objectives.
- (2) This is the first time that the fluid model is employed to solve multi-objective dynamic optimization problems, which extends the traditional state features and scheduling rules.
- (3) Compared to traditional single-stage training method, the proposed two-stage training method improves the stability of the training process.

The subsequent sections of this work are structured as follows. [Section 2](#) is the literature review. The problem description of the MOMDFJSP is discussed in [Section 3](#). The proposed DRL-based dynamic scheduling method is described in [Section 4](#). [Section 5](#) provides the results and analyses. Finally, conclusions and future recommendations are provided in [Section 6](#).

2. Literature review

Due to the scarcity of literature on DFJSP with multiplicity characteristics, we reviewed the literature on classical DFJSP. DFJSPs are a common problem in practical industrial systems. Researchers have proposed various optimization approaches to solve DFJSP, mainly including metaheuristic-based and deep reinforcement learning-based methods. Because the complexity and real-time requirements of DFJSP make it difficult for exact and heuristic algorithms to meet these intricate requirements and obtain acceptable solutions [8]. Moreover, considering the lack of literature on using fluid models to solve the FJSP, we analyze the application of fluid models in JSP to unveil its potential application value in solving the FJSP.

2.1. Metaheuristic methods for DFJSP

Numerous metaheuristic algorithms have been proposed for solving DFJSP. Gholami and Zandieh [9] introduced a simulated genetic algorithm adept at handling simulated random failures. Through the integration of right-shift scheduling and rescheduling techniques, they achieved optimal solutions, establishing this method as a crucial approach in subsequent research. Researchers have enhanced genetic algorithms, including improvements in initialization and search strategies [10]. An upgraded version of NSGA-II has been proposed for single-machine failure problems [11], considering factors like machine production status, failure probability, and floating-point time for machine failures. Other improved metaheuristic algorithms have also been employed for single-machine failure issues [12]. For instance, Wei, et al. [13] proposed an enhanced cuckoo migration algorithm, while Buddala and Mahapatra [14] designed a teacher-learning optimization algorithm. Teymourifar, et al. [15] used gene expression programming rules to extract scheduling rules. Ghaleb, et al. [16] considered real-time joint optimization of maintenance and production plans in intelligent manufacturing systems, addressing dynamic events such as new job arrivals, changes in due dates, machine degradation, and failures in dynamic FJSP. Lv, et al. [17] combined rescheduling with process planning for energy-efficient production and process flexibility, transforming the problem into a mixed-integer programming model. Although these works successfully solve the DFJSP, the performance of metaheuristic methods is significantly affected by the computation time. Therefore, most literature overlooks the impact of rescheduling time. In real-time online scheduling, the rescheduling time is very short, and metaheuristic methods can't achieve efficient responses in such a short time [8].

2.2. Deep reinforcement learning methods for DFJSP

With recent developments in DRL, researchers have extensively

Table 1

The literature on the DFJSP with the application of DRL.

Papers	Dynamic events	Objectives	Algorithms
Zhao et al. [24]	Machine breakdown	Total tardiness	Q-learning
Baer et al. [25]	Machine breakdown	Makespan	DQN
Luo [26]	Job insertion	Total tardiness	DDQN
Palombarini and Martínez [27]	New order arrival and machine breakdown	Makespan	PPO
Yang and Xu [28]	Job insertion	Makespan	A2C
Chang et al. [29]	Job insertion	Total tardiness	DDQN
Liu et al. [30]	Job insertion	Makespan	DDQN
Gui et al. [31]	New order arrival	Total tardiness	DDPG
Zhang et al. [32]	Process time change	Makespan	PPO
Lang et al. [33]	Job insertion	Makespan and total tardiness	Multi-agent DQN
Luo et al. [34]	Job insertion and machine breakdown	Total tardiness, average machine utilization rate, and variance of machine workload	HMAPPO
Luo et al. [35]	Job insertion	Total weighted tardiness and average machine utilization rate	TDQN
Li et al. [36]	Job insertion and machine breakdown	Makespan and total energy consumption	HDQN
Wang et al. [37]	New order arrival, job cancel, machine breakdown, and machine maintenance	Makespan, average machine utilization rate, and average tardiness	Multi-agent DQN
Wu et al. [38]	Job insertion	Makespan and total tardiness	DLDDQN

explored DRL methods for DFJSPs. Compared with traditional dispatching rules and metaheuristics, DRL-based methods offer the advantage of adapting the most appropriate actions at each rescheduling point. Scheduling methods rooted in DRL exhibit notable advantages over conventional rule-based or heuristic approaches. They demonstrate heightened efficacy in reducing production costs, enhancing processing efficiency, and showcasing superior flexibility and generalization capabilities [18]. These features make DRL particularly well-suited for multi-objective optimization and real-time dynamic scheduling in DFJSPs. To cope with these requirements, several studies have integrated DRL algorithms with deep deterministic policy gradient (DDPG) [19], proximal policy optimization (PPO) [20], Q-learning [21], deep Q network (DQN) [22], and double deep Q network (DDQN) [23]. Table 1 summarizes the literature on the DFJSP with the application of DRL.

Amidst the escalating uncertainty and complexity in manufacturing, re-scheduling methods to manage unexpected events have become a critical concern for real-time interrupt management. DRL is pivotal in adapting to dynamic environments, ensuring robustness without compromising cost-effectiveness, product quality, and delivery efficiency. Zhao, et al. [24] proposed a dual-layer Q-learning algorithm for the DFJSP with machine failures. Baer, et al. [25] introduced a DRL method for online scheduling in Flexible Manufacturing Systems (FMS). Luo [26] designed a network framework using the DDQN to solve the DFJSP with new job arrivals. Palombarini and Martínez [27] presented a DRL method leveraging high-dimensional sensory input for efficient online re-scheduling in dynamic production environments. Yang and Xu [28] pioneered intelligent scheduling and reconfiguration studies in reconfigurable assembly lines with new job arrivals, employing the actor-critic deep reinforcement learning method with excellent performance. Chang, et al. [29] proposed a DDQN-based architecture for deep reinforcement learning to solve the DFJSP with random job arrivals. Liu, et al. [30] introduced a hierarchical and distributed architecture,

training agents through the DDQN to capture the relationship between production information and scheduling objectives for real-time decisions in flexible job shop environments. Gui, et al. [31] presented a DDPG-based scheduling method for the DFJSP, designing a composite action of SDRs and continuous weight variables for a continuous rule space and SDR weight selection. Zhang, et al. [32] investigated a novel DFJSP model considering machine job processing time uncertainty, proposing a PPO-based deep reinforcement learning framework to address the issue. However, these studies employ DRL-based methods for the DFJSP that predominantly features single-objective optimization with single-layered network frameworks.

In multi-objective DFJSP literature, predominant dynamic events involve machine failures and random job arrivals, focusing on optimizing total tardiness time and maximum completion time. Lang, et al. [33] introduced a deep reinforcement learning algorithm with two DQN agents handling operation selection and machine allocation. Luo, et al. [34] devised a hierarchical multi-agent deep reinforcement learning framework, utilizing a multi-agent PPO training algorithm for partial no-wait multi-objective DFJSP. Furthermore, Luo, et al. [35] proposed an online rescheduling framework based on dual-layer DDQN for multi-objective DFJSP with random job arrivals. Li, et al. [36] addressed the DFJSP considering insufficient transportation resources, optimizing for maximum completion time and total energy consumption, and presenting a hybrid DQN solution for dynamic interference events. Wang, et al. [37] introduced a real-time processing framework using two DQN networks for online scheduling of multi-objective DFJSP, achieving notable optimization rates. Finally, Wu, et al. [38] explored DFJSP under random job arrivals, proposing a dual-layer DDQN network combining DRL with scheduling rules, optimizing for makespan and total tardiness time. Zhang, et al. [39] presented an adaptive multi-task multi-objective scheduling model using a hybrid strategy and a two-hierarchy deep reinforcement learning architecture to optimize total manufacturing time and cost in dynamic environments. These prevailing literature on multi-objective DFJSP with DRL focuses on addressing machine failures and random job arrivals, targeting the optimization of tardiness and completion times.

In addition, it is evident that hierarchical DRL is the main approach for tackling multi-objective DFJSPs. Initially proposed by Kulkarni, et al. [40], hierarchical DRL integrates value functions over different time scales with intrinsically motivated learning to improve exploration efficiency in sparse feedback environments. Subsequently, various hierarchical DRL methods have been developed for multi-objective optimization [41]. These methods typically feature a hierarchical network structure, which can be categorized into two types: the first involves an upper-level network responsible for selecting intermediate optimization objectives, while the lower-level network consists of operation and machine policy networks for operation selection and machine assignment [33,34]. The second type uses the lower-level network as a composite scheduling rule selection network, specifically for operation selection and machine allocation [35–39]. Our approach differs by using separate policy networks for different objectives. In addition, traditional frameworks often use a single-stage training approach, which can destabilize training due to synchronous optimization of the two-layer network. Therefore, we propose a two-stage training method to enhance stability.

2.3. Fluid model-based methods for high-multiplicity JSP

In current research on fluid model solutions for JSP, the main focus is on workshops characterized by high-multiplicity production. For example, Bertsimas and Gamarnik [42] proposed an asymptotically optimal algorithm that uses a fluid relaxation model and linear programming techniques, without the need for predetermined probability assumptions, and is capable of generating near-optimal scheduling solutions. Boudoukh, et al. [43] introduced a strategy for solving high-multiplicity JSP issues with fixed process routes and machine

numbers, using a fluid simulation heuristic for jobs with different processing times, and approximate solutions are found combining continuous fluid scheduling models with constant flow rate strategies. Bertsimas and Sethuraman [44] developed a fluid synchronization algorithm that provides near-optimal solutions for maximum completion times through fluid models and the concept of fair queuing. Dai and Weiss [45] proposed an online heuristic algorithm that sets safety stocks and takes the bottleneck machine as the pace, solving the problem and proving its approximation to the optimal solution based on probabilistic analysis. Bertsimas, et al. [46] designed an algorithm that minimizes the total holding costs through a fluid relaxation model and is asymptotically optimal as the multiplicity grows linearly, outperforming common heuristic methods in medium and large batch JSP. Nazarathy and Weiss [47] introduced a new heuristic approach, also discussing its asymptotic optimality. Masin and Raviv [48] developed a polynomial-time asymptotic approximation algorithm based on the linear relaxation of a time-indexed mixed integer model, introducing an effective heuristic for solving. Gu, et al. [49] proposed a TVSA algorithm based on a virtual problem and corresponding virtual schedule, verifying through performance analysis that the gap between its solution and the optimal solution is of a constant order, demonstrating the algorithm's asymptotic optimality. Gu, et al. [50] incorporated a fluid relaxation model for deterministic problems, proposed a fluid strategy, and proved the asymptotic optimality of the solutions produced by this strategy in comparison to the optimal solutions.

These existing literature shows the fluid model's ability to achieve near-optimal solutions and exhibit strong potential in global optimization when dealing with high-multiplicity JSPs. However, their effectiveness is significantly constrained by the size of the multiplicity, rendering them unsuitable for workshop scheduling in low-multiplicity production scenarios. In addition, compared to JSP, the FJSP is more complex, involving not only the sequencing of operations but also the allocation of machines, which makes it more difficult and challenging to develop fluid models and their solution strategies for the FJSP. Therefore, this paper develops the fluid model specifically for the FJSP that leverages their global optimization capabilities. The proposed integration of fluid models with deep reinforcement learning is suitable for solving FJSPs ranging from low to high multiplicity, thereby enhancing the practical application potential of fluid models in actual production.

3. Problem description

The MOMDFJSP consists of M machines ($m = 1, 2, \dots, M$), S orders with random arrival times ($s = \{1, 2, \dots, S\}$), and R job types ($r = \{1, 2, \dots, R\}$), where job type r consists of J_r operations. All jobs are categorized into R distinct types. In the same job, operations follow a specified sequence of precedence. An operation may be executed on multiple machines, with varying processing times and cutting powers across different machines. It is important to note that each machine is characterized by two attributes: processing power and idle power, with the latter being a constant value. In the event of a machine malfunction, it ceases to consume energy.

The orders s arrive randomly, containing N_{sr} jobs for job type r . The number of jobs for a specific job type r in order s is called its multiplicity ($N_{sr} > 1$). In other words, there are two or more jobs for one job type. In an actual manufacturing environment, the arrival of new orders containing multiple jobs for each job type is assumed to follow a Poisson distribution [51], which implies that the time interval between the arrivals of new orders follows an exponential distribution [52]. Each machine m might break down randomly. The start time of the k th breakdown on machine m is represented as bs_{mk} . The repair time for breakdown machine m is represented as bt_{mk} . To manage the MOMDFJSP's complexity, several assumptions are employed in problem-solving approaches:

- (1) All machines are limited to processing only one operation at any given time.
- (2) Operations are non-preemptive, meaning they can't be interrupted.
- (3) Operations follow a strict order, and one can't begin until the previous operation is performed.
- (4) Setup and transportation time are not considered.
- (5) If a machine breakdown interrupts an operation, it should be resumed immediately on the same machine after repair.

Here are the notations:

Notation	Description
Indexes:	
s	Orders, $s = 1, 2, \dots, S$.
m	Machines, $m = 1, 2, \dots, M$.
r, r'	Job types, $r, r' = 1, 2, \dots, R$.
n, n'	Jobs belonging to job types r and r' , $n = 1, 2, \dots, N_r, n' = 1, 2, \dots, N_{r'}$.
j, j'	Operations of job types r and r' , $j = 1, 2, \dots, J_r, j' = 1, 2, \dots, J_{r'}$.
Parameters:	
R	Total number of job types.
M	Total number of machines.
M^+	The sufficiently large positive number.
J_r	Number of operations of job type r .
J_m	n^{th} job of job type r .
o_{mj}	j^{th} operation of the n^{th} job of job type r .
o_j	j^{th} operation of job type r .
t_{rjm}	The processing time of operation o_j on machine m .
p_{rjm}	The power consumed by processing operation o_j on machine m per unit of time.
pe_{rjm}	The energy consumption of operation o_j on machine m , then $pe_{rjm} = t_{rjm}p_{rjm}$.
p_m^{idle}	The idle power of machine m .
Sets:	
M_m	Set of machines, $M_m = \{1, 2, \dots, M\}$.
M_j	Subset of machines that can process operation o_j .
R_r	Set of job types, $R_r = \{1, 2, \dots, R\}$.
O_r	Set of operations for job type r , $O_r = \{1, 2, \dots, J_r\}$.
O_{Rj}	Set of operations for all job types, $O_{Rj} = \{o_j r \in R_r, j \in O_r\}$.
O_m^{Rj}	Set of operations that can be processed by machine m , $O_m^{Rj} = \{o_j r \in R_r, j \in O_r, m \in M_j\}$.
Variables:	
PE	Total processing energy consumption.
IE	Total idle energy consumption.
S	Number of new orders.
A_s	Arrival time of order s ; order $s = 1$ is the first order, and $A_1 = 0$.
D_s	Due date of order s .
d_m	Due date of job J_m ; $d_m = D_s$ if $z_{ms} = 1$.
N_{sr}	Total number of jobs for job type r in order s .
N_s	Total number of jobs in order s , $N_s = \sum_{r=1}^R N_{sr}$.
N_r	Total number of jobs for job type r in all orders, $N_r = \sum_{s=1}^S N_{sr}$.
N	Number of jobs in all orders, $N = \sum_{s=1}^S N_s$.
K_m	Total number of breakdowns for machine m .
bs_{mk}	Start time of the k th breakdown for machine m .
bt_{mk}	Repair time of the k th breakdown for machine m .
Decision variables:	
st_m	Start time of machine m .
ct_m	Completion time of machine m .
st_{mj}	Start time of operation o_{mj} .
ct_{mj}	Completion time of operation o_{mj} .
x_{mjim}	$x_{mjim} = \begin{cases} 1, & \text{if } o_{mj} \text{ is assigned on machine } m. \\ 0, & \text{otherwise.} \end{cases}$
$y_{mjirnj'}$	$y_{mjirnj'} = \begin{cases} \&1, & \text{if } o_{mj} \text{ is a predecessor of } o_{r'nj'}. \\ \&-1, & \text{if } o_{mj} \text{ is a successor of } o_{r'nj'}. \end{cases}$
z_{ms}	$z_{ms} = \begin{cases} \&1, & \text{if job } J_m \text{ belongs to orders.} \\ \&0, & \text{otherwise.} \end{cases}$

We simultaneously consider three objectives: makespan, total tardiness time, and total energy consumption as follows:

$$C_{\max} = \max\{ct_{mr} | r \in R_r, n \in I_r\} \quad (1)$$

$$T_{\max} = \sum_{r=1}^R \sum_{n=1}^{N_r} \max(ct_{mr} - d_m, 0) \quad (2)$$

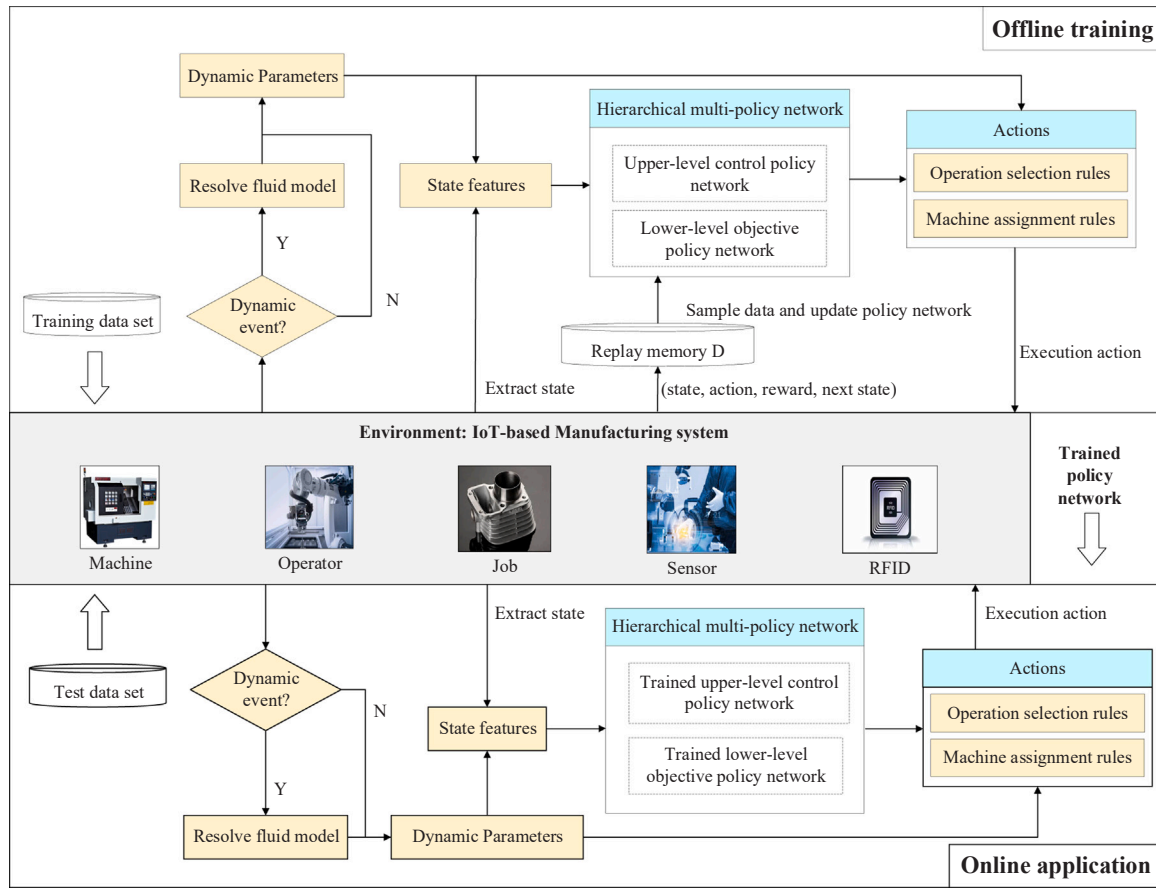


Fig. 1. Data-driven hierarchical multi-policy deep reinforcement learning framework.

$$E_{\max} = PE + IE \quad (3)$$

In Eq. (3), total processing energy consumption PE and total idle energy consumption IE are calculated as:

$$PE = \sum_{m=1}^M \sum_{r=1}^R \sum_{j=1}^{J_r} \sum_{n=1}^{N_r} t_{rjm} x_{rjm} p_{rjm} \quad (4)$$

$$IE = \sum_{m=1}^M \left((ct_m - st_m) - \sum_{r=1}^R \sum_{j=1}^{J_r} \sum_{n=1}^{N_r} t_{rjm} x_{rjm} - \sum_{k=1}^{K_m} bt_{mk} \right) p_m^{idle} \quad (5)$$

Subject to:

$$st_{rnj} \geq 0, \forall r, n, j \quad (6)$$

$$\sum_{m \in M_j} x_{rjm} = 1, \forall r, n, j \quad (7)$$

$$\sum_{s=1}^S z_{ms} = 1, \forall r, n \quad (8)$$

$$st_m \leq st_{rnj} x_{rjm} + (1 - x_{rjm}) M^+, \forall r, n, j, m \quad (9)$$

$$ct_m \geq ct_{rnj} x_{rjm} - (1 - x_{rjm}) M^+, \forall r, n, j, m \quad (10)$$

$$(ct_{rnj} - t_{rjm} - ct_{rnj-1}) x_{rjm} \geq 0, \forall r, n, j, m \quad (11)$$

$$(ct_{rnj} - t_{rjm} - A_s) z_{ms} x_{rjm} \geq 0, \forall r, n, j, m, s \quad (12)$$

$$(ct_{r'n'j'} - t_{r'n'm} - ct_{rnj}) x_{rjm} x_{r'n'j'm} y_{rnjr'n'j'} \geq 0, \forall r, r', n, n', j, j', m \quad (13)$$

$$(ct_{rnj} - t_{rjm} - ct_{r'n'j'}) x_{rjm} x_{r'n'j'm} (1 - y_{rnjr'n'j'}) \geq 0, \forall r, r', n, n', j, j', m \quad (14)$$

$$(st_{rnj} + t_{rjm} - bs_{mk}) x_{rjm} (st_{rnj} - bs_{mk}) \leq 0 \quad (15)$$

$$(st_{rnj} + t_{rjm} + bt_{mk} - ct_{rnj}) x_{rjm} = 0, \forall r, n, j, m, k \quad (15)$$

Eq. (1) represents the objective of maximum completion time, Eq. (2) denotes the total tardiness time objective, and Eq. (3) is the total energy consumption objective. Eq. (4) is the total processing energy consumption, while Eq. (5) is the total idle energy consumption. Eq. (6) means that the start time of an operation is nonnegative. Eq. (7) indicates that each operation can only be processed by one machine from the set of available machines. Eq. (8) represents that each job can only belong to one order. Eqs. (9) and (10) impose constraints on the start and completion times of machines. Eq. (11) ensures priority constraints for operations belonging to the same job. Eq. (12) ensures that a job can only be processed after it arrives. Eqs. (13) and (14) specify that each machine can process at most one operation at the same time. Eq. (15) states that if an operation is interrupted during processing due to a machine breakdown, the operation continues on the repaired machine after the repair is complete.

4. Methodology

The proposed data-driven hierarchical multi-policy DRL scheduling framework is designed to address the MOMDFJSP effectively. It consists of two main steps: offline training and online application, as outlined in Fig. 1. Dynamic scheduling problems are particularly suited for reinforcement learning due to their evolving nature. During the offline training phase, the policy network interacts with the environment to

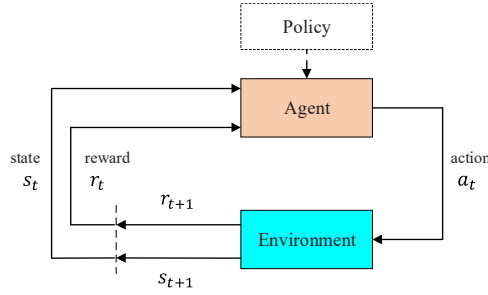


Fig. 2. The interaction learning process between the agent and environment.

accumulate historical data. Subsequently, the network leverages the DRL method and past decision-making data to optimize its parameters, learning to select suitable scheduling rules based on state data. The effectiveness of DRL solutions is significantly influenced by the quality of state features and scheduling rules. To enhance these aspects, we integrate a fluid model that assists in designing state features and scheduling rules, thereby improving the state representation and decision-making actions of the reinforcement learning system.

In the online application step, the trained policy network is deployed for real-time online scheduling. Considering that offline training may not encompass all possible real-world scenarios, we intend to periodically acquire real-world instances and high-quality scheduling schemes. Then, we retrain and refine the trained policy network model through a process termed self-learning. This self-learning approach can be implemented in two ways: (1) Use labeled high-quality scheduling schemes to adjust the parameters of the policy network model, which is consistent with supervised learning. (2) Apply DRL techniques to train the neural network further using real-world instances.

The proposed framework triggers the solution of the fluid model by real-time monitoring of dynamic events in the workshop, ensuring that the fluid model accurately reflects instantaneous changes in the production environment. In addition, by combining the dynamic parameters of the fluid model with the real-time processing conditions of the manufacturing system, state features and scheduling rules that interact with the dynamic parameters of the fluid model are designed to enhance state representation capability and the quality of decision-making actions. The proposed framework for MOMDFJSP uses the fine-grained, continuous system state information provided by the fluid model as input for learning and decision-making. This not only promotes precise characterization of the manufacturing process, but also enhances the adaptability and optimization performance of the scheduling rules. Consequently, the proposed framework fully leverages the advantages of fluid models in describing system dynamics and harnessing the capabilities of deep reinforcement learning for decision optimization, thereby aiming to enhance the performance of scheduling systems.

4.1. Background of deep reinforcement learning

This section commences with a discussion on the foundational principles of the classical reinforcement learning framework. Subsequently, we introduce the principles of maximum entropy reinforcement learning-soft actor-critic (SAC) algorithm, unveiling its potential advantages in addressing complex multi-objective optimization problems.

4.1.1. Reinforcement learning

The foundational structure for addressing reinforcement learning (RL) problems is represented as the Markov Decision Process (MDP), which provides a classical framework for sequential decision-making processes. In the MDP, the agent is responsible for learning and executing decisions, while the environment encompasses all external factors that the agent interacts with. As shown in Fig. 2. At each discrete time step t , the agent (policy network) assesses the current state of the

environment, denoted by s_t , and chooses an action a_t in response. After the action, the agent receives a reward r_t and transitions to a new state s_{t+1} . This interaction generates a temporal sequence: $\{s_0, a_0, r_0, s_1, a_1, r_1, s_2, \dots\}$. The agent's objective is to discover the most effective policy π^* that maximizes the expected sum of rewards accumulated by adhering to this policy from any given state.

4.1.2. Soft actor-critic algorithm

The SAC algorithm is an entropy-based maximization method [53]. By incorporating entropy into the objective function, the SAC significantly enhances its exploration capabilities and robustness, striving to achieve the balance between reward and entropy (i.e., the randomness of the policy). The algorithm encourages higher rewards for actions executed with greater randomness, thereby averting premature convergence to suboptimal policy. A higher entropy value leads to more exploration, thus avoiding policy convergence to local optima. Furthermore, the multi-objective optimization problems typically involve complex decision and objective spaces. The maximum entropy characteristic of SAC can maintain the balance between multiple objectives. The optimal policy of SAC is represented as Eq. (16). In Eq. (16), π^* represents the optimal policy. $\pi(s_t)^T$ represents the probabilities of choosing each discrete action. α is the entropy coefficient. $H(\pi(s_t)) = \mathbb{E}[-\log(\pi(s_t))]$ denotes the entropy value.

$$\pi^* = \operatorname{argmax}_{\pi} \pi(s_t)^T \left[\sum_{t=0}^{\infty} \gamma^t r_t + \alpha H(\pi(s_t)) \right] \quad (16)$$

The SAC includes five networks: two state-action value networks Q_{ϕ_1} and Q_{ϕ_2} , two target state action value networks $Q_{\phi'_1}$ and $Q_{\phi'_2}$, and one policy network π_{θ} . The parameters ϕ_1 and ϕ_2 of two state action value networks are optimized using gradient descent, as shown in Eq. (17). In Eq. (17), B represents the batch size of the samples from the reply buffer. y_t is the target value, calculated as in Eq. (18).

$$\nabla_{\phi_i} J(\phi_i) = B^{-1} \sum \nabla_{\phi_i} (Q_{\phi_i}(s_t, a_t) - y_t)^2, i \in \{1, 2\} \quad (17)$$

$$y_t = r_t + \gamma \pi_{\theta}(s_t)^T \left(\operatorname{argmin}_{i=1,2} Q_{\phi'_i}(s_{t+1}) - \alpha \log(\pi_{\theta}(s_{t+1})) \right) \quad (18)$$

The target state action value network employs a soft update mechanism, introducing a hyperparameter τ . This mechanism computes a weighted average between the old target network parameters and the new corresponding network parameters, which is then assigned to the target network. This updating method allows for gradual changes to the target network parameters, significantly enhancing the stability of learning. The specific update method is illustrated in the following Eq. (19). The parameters θ of policy network π_{θ} are optimized with Eq. (20).

$$\phi'_i \leftarrow \tau \phi_i + (1 - \tau) \phi'_i, i \in \{1, 2\} \quad (19)$$

$$\nabla_{\theta} J(\theta) = B^{-1} \sum \nabla_{\theta} \left(\pi_{\theta}(s_t)^T \left(\alpha \log(\pi_{\theta}(s_t)) - \operatorname{argmin}_{i=1,2} Q_{\phi_i}(s_t) \right) \right) \quad (20)$$

In the traditional approach, we should use a constant entropy regularization coefficient α . However, this constant value may not be appropriate due to the variability of the rewards, which may lead to training instabilities. Therefore, it is essential to implement a mechanism for the automatic adjustment of α . An adaptive entropy loss function is employed for dynamically adjusting α , as shown in the following Eq. (21). In this context, $H_0 = -\dim(A)$ symbolizes the desired entropy value, with A representing the number of discrete actions.

$$\nabla_{\alpha} J(\alpha) = \pi_{\theta}(s_t)^T \nabla_{\alpha} [-\alpha \log(\pi_{\theta}(s_t)) - \alpha H_0] \quad (21)$$

4.2. Fluid model

The fluid model, premised on the principles of fluid dynamics, abstracts and analogizes the processing activities and queuing phenomena in manufacturing systems to fluid flow. This model assumes that the

Table 2

The notation and description of fluid model.

Notation	Description
Parameters:	
e_{mj}	Processing rate of operation o_{rj} on machine m , $e_{mj} = \frac{1}{t_{rjm}}$.
Variables:	
t_s	The time for resolving the fluid model is prompted by the arrival of new orders or machine breakdowns.
u_{mj}	Time proportion allocated by machine m for processing operation o_{rj} in the fluid model.
e_{rj}	Total processing rate of operation o_{rj} across all machines in the fluid model, $e_{rj} = \sum_{m=1}^M e_{mj} u_{mj}$.
ℓ_{rj}^f	Completion time of operation o_{rj} in the fluid model.
$q_{rj}(t)$	Number of jobs in operation stage o_{rj} at time t in the fluid model.
$Q_{rj}^+(t)$	Total number of jobs in which operation o_{rj} has not been processed at time t in the fluid model.
$q_{mj}^+(t)$	Total number of jobs in which operation o_{rj} has not been processed by machine m at time t in the fluid model.
$I_{rj}(t)$	Set of jobs in operation stage o_{rj} at time t in the manufacturing system.
$I_{rj}^+(t)$	Set of jobs in which operation o_{rj} has not been processed at time t in the manufacturing system; $I_{rj}^+(t) = \sum_{j=1}^j I_{rj}(t)$.
$Q_{rj}(t)$	Number of jobs in operation stage o_{rj} at time t in the manufacturing system; $Q_{rj}(t) = I_{rj}(t) $.
$Q_{rj}^+(t)$	Total number of jobs in which operation o_{rj} has not been processed at time t in the manufacturing system; $Q_{rj}^+(t) = I_{rj}^+(t) $.
$Q_{mj}^+(t)$	Total number of jobs in which operation o_{rj} has not been processed by machine m at time t in the manufacturing system.

fluid (jobs in the manufacturing systems) flows through the system in a continuous manner. This assumption aids in simplifying the complex discrete production processes. By such means, the fluid model captures essential attributes of the manufacturing system, such as resource allocation, and can be employed to forecast performance indicators like system completion time.

The proposed fluid model is a linear programming model. Therefore, the fluid model can be easily solved by CPLEX. The fluid model solution is called the fluid solution. The discrete processing of an operation o_{rj} can be denoted as a continuous fluid flow. The processing time of machine m can be allocated to its processable operations o_{rj} ($\forall o_{rj} \in O_m^{RJ}$). The machine allocates a fixed proportion of time to different operations in a unit. Based on the fluid model, the time proportion u_{mj} that machine m allocates to its processable operation o_{rj} in a unit of time can be obtained. The fluid solution obtained by solving the fluid model represents the time proportions $\{u_{mj}|m \in M_m, o_{rj} \in O_m^{RJ}\}$ (u_{mj} is a set of real numbers between 0 and 1) for each machine to process each operation. For example, if machine m can process operations $\{o_{11}, o_{12}, o_{21}\}$, the optimal time proportions for machine m to process each operation obtained by solving the fluid model are $\{u_{m11} = 0.3, u_{m12} = 0.4, u_{m21} = 0.2\}$. These time proportions can be interpreted as follows: in each unit of time, machine m spends 0.3 units of time processing operation o_{11} , 0.4 units of time processing operation o_{12} , and 0.2 units of time processing operation o_{21} .

In response to dynamic events, the fluid model is resolved by the current state of the manufacturing system to enhance the accuracy of its representation, particularly in scenarios involving machine breakdowns and the arrival of new orders. This strategy of dynamic resolution enables rapid model adjustments, accurately reflecting the real-time status of production processes. Consequently, this enhances the model's adaptability and predictive accuracy in uncertain operational environments. The fluid model can be described as follows Table 2.

In the fluid model, $I_{rj}(t)$, $I_{rj}^+(t)$, $Q_{rj}(t)$, $Q_{rj}^+(t)$, and $Q_{mj}^+(t)$ represent the processing status properties of operations in the manufacturing system, which can be obtained in real-time based on the actual processing status of the manufacturing system. $q_{rj}(t)$, $q_{rj}^+(t)$, and $q_{mj}^+(t)$ represent the fluid properties of operation o_{rj} , which can be calculated based on the fluid

solution $\{u_{mj}|m \in M_m, o_{rj} \in O_m^{RJ}\}$ and Eqs. (22) - (26).

The proposed fluid model satisfies Eqs. (22) - (30). Eq. (22) states that at time t , $q_{rj}(t)$ is the number of jobs in operation stage o_{rj} at time t_s plus the number of jobs transitioned from operation stage o_{rj-1} to operation stage o_{rj} in $t \in [t_s, t)$ minus the processed number of jobs in operation stage o_{rj} in the time interval $[t_s, t)$. $q_{rj}(t_s)$ is the number of jobs in operation stage o_{rj} at time t_s in the manufacturing system, denoted as $Q_{rj}(t_s)$.

$$q_{rj}(t) = \begin{cases} q_{rj}(t_s) + t(e_{rj-1} - e_{rj}), & \forall o_{rj} \in O_{RJ}, j \geq 2 \\ q_{rj}(t_s) - te_{rj}, & \forall o_{rj} \in O_{RJ}, j = 1 \end{cases} \quad (22)$$

$q_{rj}^+(t)$ can be calculated by Eq. (23). $q_{rj}^+(t)$ is the number of jobs that operation o_{rj} has not been processed at time t_s minus the jobs handled in operation stage o_{rj} in the time interval $[t_s, t)$. $q_{rj}^+(t_s)$ is the total number of jobs for which operation o_{rj} has not been processed at time t in the manufacturing system at time t_s , denoted as $Q_{rj}^+(t_s)$.

$$q_{rj}^+(t) = q_{rj}^+(t_s) - (t - t_s)e_{rj}, \forall o_{rj} \in O_{RJ} \quad (23)$$

Eq. (24) calculates the total number of jobs where operation o_{rj} remains unprocessed by machine m at time t_s . The parameter $\frac{e_{mk_{rj}} u_{mk_{rj}}}{e_{k_{rj}}}$ represents the contribution ratio of machine m to the total processing rate of operation o_{rj} . $q_{mj}^+(t)$ is calculated by Eq. (25). The completion time for

Table 3

The dynamic parameters of the fluid model.

Parameters	Description
$gap_{rj}(t)$	At time t , the gap between the total number of jobs for which operation o_{rj} has not been processed in the manufacturing system and the fluid model, then $gap_{rj}(t) = Q_{rj}^+(t) - q_{rj}^+(t)$, $\forall o_{rj} \in O_{RJ}$. $Q_{rj}^+(t)$ is related to the current processing status of the manufacturing system. $Q_{rj}^+(t) = Q_{rj}^+(t_s) - Q_{rj}^-(t)$, and $Q_{rj}^-(t)$ represents the cumulative number of jobs in which operation o_{rj} has been processed in the manufacturing system in time interval $[t_s, t)$.
$gap_{mj}(t)$	At time t , the gap between the total number of jobs whose operation o_{rj} has not been processed by machine m in the manufacturing system and the total number of jobs for which operation o_{rj} has not been processed by machine m in the fluid model, then $gap_{mj}(t) = Q_{mj}^+(t) - q_{mj}^+(t)$, $\forall o_{rj} \in O_{RJ}, m \in M_m$. $Q_{mj}^+(t) = Q_{mj}^+(t_s) - Q_{mj}^-(t)$, and $Q_{mj}^-(t)$ represents the cumulative number of jobs in which operation o_{rj} has been processed by machine m in the manufacturing system in time interval $[t_s, t)$.
$gap_m(t)$	The mean value of $gap_{mj}(t)$ for machine m , defined as $gap_m(t) = \frac{\sum_{o_{rj} \in O_m^{RJ}} gap_{mj}(t)}{ O_m^{RJ} }$.
$gap_{rj}^{ratio}(t)$	Ratio of $gap_{rj}(t)$ to $Q_{rj}^+(t_s)$, where $gap_{rj}^{ratio} = \frac{gap_{rj}(t)}{Q_{rj}^+(t_s)}$, $\forall o_{rj} \in O_{RJ}$.
O_{m-f}^{RJ}	Set of selectable operations for machine m in the fluid model. It is defined as $O_{m-f}^{RJ} = \{o_{rj} o_{rj} \in O_m^{RJ}, u_{mj} > 0\}$, where $u_{mj} > 0$ indicates that machine m has a nonzero proportion of processing time allocated to operation o_{rj} . If $u_{mj} = 0$, machine m does not allocate any processing time to operation o_{rj} , and $o_{rj} \notin O_{m-f}^{RJ}$.
$O_{idle-f}^{RJ}(t)$	Set of selectable operations for idle machines in the fluid model. It is defined as $O_{idle-f}^{RJ}(t) = \{o_{rj} m \in M_{idle}(t), o_{rj} \in O_{m-f}^{RJ}, Q_{rj}(t) > 0\}$, where $Q_{rj}(t) > 0$ indicates that there are jobs in operation stage o_{rj} , making operation o_{rj} selectable.
ℓ_{rj}^f	Processing rate of operation o_{rj} in the fluid model, then $\ell_{rj}^f = \frac{1}{e_{rj}}$.
M_{rj}^f	The available machine set for operation o_{rj} in the fluid model, defined as $M_{rj}^f = \{m \forall m \in M_{rj}, u_{mj} > 0\}$, where $u_{mj} > 0$ indicates that machine m is assigned a nonzero proportion of the processing time for operation o_{rj} in the fluid model. If $u_{mj} = 0$, machine m is not assigned any processing time for operation o_{rj} in the fluid model, i.e., $m \notin M_{rj}^f$.
$M_{idle-f}^{RJ}(t)$	The available idle machine set for operation o_{rj} in the fluid model, defined as $M_{idle-f}^{RJ}(t) = \{m m \in M_{rj}^f \cap M_{idle}(t)\}$, where $M_{idle}(t)$ is the set of idle machines at decision point t .

Table 4

The notation and description of state features.

Notation	Description
$s(t)$	Total number of orders that have arrived.
$N_r(t)$	Total number of jobs for job type r among the arrived orders, where $N_r(t) = \sum_{s=1}^{s(t)} N_{sr}$.
$N_{rj}(t)$	Total number of operations o_{rj} among the arrived orders, where $N_{rj}(t) = N_r(t)$.
$N_{rj}^-(t)$	Total number of operations o_{rj} that have been processed.
$CT_m(t)$	Completion time of machine m .
$CT_M^{ave}(t)$	Average completion time of all machines, where $CT_M^{ave}(t) = \frac{\sum_{m=1}^M CT_m(t)}{M}$.
$CRO_{rj}(t)$	Completion rate of operation o_{rj} , where $CRO_{rj}(t) = \frac{N_{rj}^-(t)}{N_{rj}(t)}$.
$M_{idle}(t)$	The actual available idle machine set for operation o_{rj} in the manufacturing system, defined as $M_{rj}^{idle}(t) = \{m m \in M_{rj} \cap M_{idle}(t)\}$.
$U_m(t)$	The utilization rate of machine m .
$O_{idle}^{RJ}(t)$	Set of actual selectable operations for idle machines in the manufacturing system. It is defined as $O_{idle}^{RJ}(t) = \{o_{rj} m \in M_{idle}(t), o_{rj} \in O_m^{RJ}, Q_{rj}(t) > 0\}$.

operation o_{rj} of job type r is shown in Eq. (26).

$$q_{mrj}^+(t_s) = q_{rj}^+(t_s) \frac{e_{mrj} u_{mrj}}{e_{rj}}, \forall o_{rj} \in O_{RJ}, m \in M_{rj} \quad (24)$$

$$q_{mrj}^+(t) = q_{mrj}^+(t_s) - (t - t_s) e_{mrj} u_{mrj}, \forall o_{rj} \in O_{RJ}, m \in M_{rj} \quad (25)$$

$$c_{rj}^f = \frac{q_{rj}^+(t_s)}{e_{rj}}, \forall o_{rj} \in O_{RJ} \quad (26)$$

Eq. (27) states that the utilization rate of each machine can't exceed 100 %. Eq. (28) indicates that if $q_{rj}(t_s) = 0$, then the total processing rate of operation o_{rj} is no greater than its arrival rate. This makes $q_{rj}(t) \geq 0$ true, which guarantees that the fluid solution is feasible. Eq. (29) indicates the ranges of the decision variables.

$$\sum_{o_{rj} \in M_{rj}} u_{mrj} \leq 1, \forall m \in M_m \quad (27)$$

$$e_{rj} \leq e_{rj-1}, \forall k_{rj} \in F, j \geq 2, q_{rj}(t_s) = 0 = 0 \quad (28)$$

$$u_{mrj} \in [0, 1], \forall o_{rj} \in O_{RJ}, m \in M_{rj} \quad (29)$$

The fluid model aims to minimize the maximum completion time of all operations at each rescheduled point t_s , as shown in Eq. (30).

$$\min C_s^f = \min \left(\max \{ c_{rj}^f | o_{rj} \in O_{RJ} \} \right) \quad (30)$$

4.3. Fluid Model-based dynamic parameters

Based on the above fluid model, we obtain the fluid solution. Nonetheless, the fluid solution only indicates the time proportion allocated to a specific operation o_{rj} on machine m , without specifying the detailed job sequence for each machine. Moreover, shop floor dynamic scheduling is a discrete production environment process involving operation sequence and machine assignment problems. Therefore, fluid model-based dynamic parameters are extracted to design state features, operation selection rules, and machine assignment. These dynamic parameters are evolving over time in fluid models. The extracted dynamic parameters are shown in Table 3.

4.4. State features

State features play a crucial role as inputs to the neural network for the deep reinforcement learning agent to make suitable decisions at each decision point t in the production process. To enhance the performance

of the proposed method, fifteen state features that integrated fluid model-based dynamic parameters to represent a decision point are extracted. Before providing detailed definitions of the state features at each decision point t , we list the important symbols at each decision point t used as follows Table 4:

1) The dynamic event types.

$$D_{random}(t) = \begin{cases} 0.5, \text{order arrival} \\ 1.0, \text{machine breakdown} \\ 0, \text{without dynamic events} \end{cases} \quad (31)$$

2) The average utilization rate of machines.

$$U_{ave}(t) = \frac{\sum_{m=1}^M U_m(t)}{M} \quad (32)$$

3) The standard deviation of machine utilization rate.

$$U_{std}(t) = \sqrt{\frac{\sum_{m=1}^M (U_m(t) - U_{ave}(t))^2}{M}} \quad (33)$$

4) The ratio of

$$\left| O_{idle-f}^{RJ}(t) \right| \left| O_{idle}^{RJ}(t) \right|$$

$$Ratio_{idle}^{RJ}(t) = \frac{\left| O_{idle-f}^{RJ}(t) \right|}{\left| O_{idle}^{RJ}(t) \right|} \quad (34)$$

5) The standard deviation of themachine completion times $CT_M^{std}(t)$.

$$CT_M^{std}(t) = \sqrt{\frac{\sum_{m=1}^M (CT_m(t) - CT_M^{ave}(t))^2}{M}} \quad (35)$$

6) The average operation completion rate $CRO_{ave}(t)$.

$$CRO_{ave}(t) = \frac{\sum_{r=1}^R \sum_{j=1}^{J_r} CRO_{rj}(t)}{\sum_{r=1}^R J_r} \quad (36)$$

7) The standard deviation of the operation completion rate $CRO_{std}(t)$.

$$CRO_{std}(t) = \sqrt{\frac{\sum_{r=1}^R \sum_{j=1}^{J_r} (CRO_{rj}(t) - CRO_{ave}(t))^2}{\sum_{r=1}^R J_r}} \quad (37)$$

8) The mean of $gap_{rj}^{ratio}(t)$.

$$GAP_{ave}(t) = \frac{\sum_{r=1}^R \sum_{j=1}^{J_r} gap_{rj}^{ratio}(t)}{\sum_{r=1}^R J_r} \quad (38)$$

9) The standard deviation of $gap_{rj}^{ratio}(t)$.

$$GAP_{std}(t) = \sqrt{\frac{\sum_{r=1}^R \sum_{j=1}^{J_r} (gap_{rj}^{ratio}(t) - GAP_{ave}(t))^2}{\sum_{r=1}^R J_r}} \quad (39)$$

10) The mean of $gap_m(t)$.

$$GAPM_{ave}(t) = \frac{\sum_{m=1}^M gap_m(t)}{M} \quad (40)$$

11) The standard deviation of $gap_m(t)$.

$$GAPM_{std}(t) = \sqrt{\frac{\sum_{m=1}^M (gap_m(t) - GAPM_{ave}(t))^2}{M}} \quad (41)$$

12) The actual operation tardiness rate $DRO_a(t)$.

$DRO_a(t)$ is calculated as the ratio of actual tardy operations to unprocessed operations. For an unprocessed operation $o_{mj} \in \{o_{mj} | \forall r \in R_r, j \in O_r, n \in I_{rj}^+(t)\}$, if $t > d_m$, where t represents the current decision point, operation o_{mj} is considered an actual tardy operation. The set of actual tardy operations can be represented as $ATO(t) = \{o_{mj} | \forall r \in R_r, j \in O_r, n \in I_{rj}^+(t), t > d_m\}$. Therefore, the actual operation tardiness rate $DRO_a(t)$ can be calculated as follows:

$$DRO_a(t) = \frac{|ATO(t)|}{\sum_{r=1}^R \sum_{j=1}^{J_r} |I_{rj}^+(t)|} \quad (42)$$

13) Estimated operation tardiness rate $DRO_e(t)$.

$DRO_e(t)$ is calculated as the ratio of estimated tardy operations to unprocessed operations. The set of estimated tardy operations in unfinished job $J_m \in \{J_m | \forall r \in R_r, j \in O_r, n \in I_{rj}(t)\}$ can be represented as $ETO_m(t) = \{o_{mj} | \forall j'' \in \{j, \dots, J_r\}, t + (i_n - 1)t_{rj}^f + \sum_{j=j}^{j''} t_{rj}^f > d_m\}$, where i_n represents that J_m is the i_n -th job in job set $I_{rj}(t)$, $t + (i_n - 1)t_{rj}^f$ represents the estimated start time of job J_m at operation stage o_{rj} , and $t + (i_n - 1)t_{rj}^f + \sum_{j=j}^{j''} t_{rj}^f$ represents the estimated completion time of subsequent operation $o_{mj''}$ for job J_m . Therefore, the estimated operation tardiness rate $DRO_e(t)$ can be calculated as follows:

$$DRO_e(t) = \frac{\sum_{r \in R_r, j \in O_r, n \in I_{rj}(t)} |ETO_m(t)|}{\sum_{r=1}^R \sum_{j=1}^{J_r} |I_{rj}^+(t)|} \quad (43)$$

14) Actual job tardiness rate $DRJ_a(t)$.

$DRJ_a(t)$ is calculated as the ratio of actual tardy jobs to unfinished jobs. The set of actual tardy jobs in operation stage o_{rj} is denoted as $ADJ_{rj}(t) = \{J_m | \forall n \in I_{rj}(t), t > d_m\}$. Therefore, the actual job tardiness rate $DRJ_a(t)$ can be calculated as follows:

$$DRJ_a(t) = \frac{\sum_{r \in R_r, j \in O_r} |ADJ_{rj}(t)|}{\sum_{r=1}^R \sum_{j=1}^{J_r} |I_{rj}(t)|} \quad (44)$$

15) Estimated job tardiness rate $DRJ_e(t)$. $DRJ_e(t)$ is calculated as the ratio of estimated tardy jobs to unfinished jobs. The set of estimated tardy jobs in the operation stage o_{rj} can be represented as $EDJ_{rj}(t) = \{J_m | \forall n \in I_{rj}(t), t + (i_n - 1)t_{rj}^f + \sum_{j=j}^{J_r} t_{rj}^f > d_m\}$, where t represents the current decision point, i_n represents that J_m is the i_n -th job in job set $I_{rj}(t)$, $t + (i_n - 1)t_{rj}^f$ represents the estimated start time of job J_m in operation stage o_{rj} , and $t + (i_n - 1)t_{rj}^f + \sum_{j=j}^{J_r} t_{rj}^f$ represents the estimated completion time of job J_m . Therefore, the estimated job tardiness rate $DRJ_e(t)$ can be calculated as follows:

Table 5

The notation and description of dispatching rules.

Notation	Description
$ct_{mj}^e(t)$	Estimated completion time of unfinished process o_{mj} in job J_m . It is defined as $ct_{mj}^e(t) = t + (i_n - 1)\bar{t}_{rj} + \sum_{j=j}^{J_r} \bar{t}_{rj}$, where i_n represents the i_n -th job in job set $I_{rj}(t)$ and $t + (i_n - 1)\bar{t}_{rj}$ represents the estimated start time of job J_m at operation stage o_{rj} .
$t_{rj}^{\min}(t)$	The minimum processing time of operation o_{rj} in the machine set $M_{rj}^{\min}(t)$, defined as $t_{rj}^{\min}(t) = \min\{t_{rjm} m \in M_{rj}^{\min}(t)\}$, $\forall o_{rj} \in O_{rj}^{\min}(t)$.
$t_{rj-f}^{\min}(t)$	The minimum processing time of operation o_{rj} in the machine set $M_{rj-f}^{\min}(t)$, defined as $t_{rj-f}^{\min}(t) = \min\{t_{rjm} m \in M_{rj-f}^{\min}(t)\}$, $\forall o_{rj} \in O_{rj-f}^{\min}(t)$.
$ETO_{RJ}(t)$	Set of estimated tardy operations in the operation set $O_{RJ}^{\min}(t)$. It is defined as $ETO_{RJ}(t) = \{o_{rj} \forall o_{rj} \in O_{RJ}^{\min}(t), \exists n \in I_{rj}^+(t), ct_{mj}^e(t) > d_m\}$.
$ATO_{RJ}(t)$	Set of actual tardy operations in the operation set $O_{RJ}^{\min}(t)$. It is defined as $ATO_{RJ}(t) = \{o_{rj} \forall o_{rj} \in O_{RJ}^{\min}(t), \exists n \in I_{rj}^+(t), t > d_m\}$.
$ET_{rj}^{\max}(t)$	Maximum estimated tardy time of operation o_{rj} in the operation set $O_{RJ}^{\min}(t)$. It is defined as $ET_{rj}^{\max}(t) = \max\{ct_{mj}^e(t) - d_m, 0\} n \in I_{rj}^+(t)\}$, $\forall o_{rj} \in O_{RJ}^{\min}(t)$, where $\max\{ct_{mj}^e(t) - d_m, 0\}$ represents the estimated tardy time of the specific unfinished operation o_{mj} that belongs to operation o_{rj} .
$AT_{rj}^{\max}(t)$	Maximum actual tardy time of operation o_{rj} in the operation set $O_{RJ}^{\min}(t)$. It is defined as $AT_{rj}^{\max}(t) = \max\{t - d_m, 0\} n \in I_{rj}^+(t)\}$, $\forall o_{rj} \in O_{RJ}^{\min}(t)$, where $\max\{t - d_m, 0\}$ represents the actual tardy time of the specific unfinished operation o_{mj} that belongs to operation o_{rj} .
$URG_{rj}(t)$	Estimated urgency level of operation o_{rj} in operation set $O_{RJ}^{\min}(t)$. It is defined as $URG_{rj}(t) = \frac{\sum_{n \in I_{rj}^+(t)} (ct_{mj}^e(t) - d_m)}{ I_{rj}^+(t) }$, $\forall o_{rj} \in O_{RJ}^{\min}(t)$.
$Tard_{rj}^{\min}(t)$	Minimum due date of operation o_{rj} in operation set $O_{RJ}^{\min}(t)$. It is defined as $Tard_{rj}^{\min}(t) = \min\{d_m n \in I_{rj}^+(t)\}$, $\forall o_{rj} \in O_{RJ}^{\min}(t)$.
$pe_{rj}^{\min}(t)$	The minimum processing energy consumption of operation o_{rj} in the operation set $O_{RJ}^{\min}(t)$ on the machine set $M_{rj}^{\min}(t)$, defined as $pe_{rj}^{\min}(t) = \min\{pe_{rjm} m \in M_{rj}^{\min}(t)\}$, $\forall o_{rj} \in O_{RJ}^{\min}(t)$.
$pe_{rj-f}^{\min}(t)$	The minimum processing energy consumption of operation o_{rj} in the operation set $O_{RJ}^{\min}(t)$ on the machine set $M_{rj-f}^{\min}(t)$, defined as $pe_{rj-f}^{\min}(t) = \min\{pe_{rjm} m \in M_{rj-f}^{\min}(t)\}$, $\forall o_{rj} \in O_{RJ}^{\min}(t)$.

$$DRJ_e(t) = \frac{\sum_{r \in R_r, j \in O_r} |EDJ_{rj}(t)|}{\sum_{r=1}^R \sum_{j=1}^{J_r} |I_{rj}(t)|} \quad (45)$$

Referring to the earlier definitions, we can derive the following state feature vector: $\mathbf{v}_t = [D_{random}(t), U_{ave}(t), U_{std}(t), Ratio_{idle}^{RJ}(t), CT_M^{std}(t), CRO_{ave}(t), CRO_{std}(t), GAP_{ave}(t), GAP_{std}(t), GAPM_{ave}(t), GAPM_{std}(t), DRO_a(t), DRO_e(t), DRJ_a(t), DRJ_e(t)]$. Finally, we define the state feature vector at decision point t as $\mathbf{s}_t = [\mathbf{v}_t, \mathbf{v}_t - \mathbf{v}_{t-1}]$.

4.5. Operation selection rules

This section defines twelve operation selection rules as available actions for the operation policy network. At each decision point t , the operation policy network selects an operation selection rule a_{ot} based on the current state $\mathbf{s}_t$, after which an unprocessed operation o_{rj} can be selected by a_{ot} . Before providing detailed definitions of the operation selection rules, we list the important symbols used as follows Table 5:

1) Operation selection rule 1

If $ETO_{RJ}(t) = \emptyset$, select the operation o_{rj} with the maximum $URG_{rj}(t)$ value from the set $O_{RJ}^{\min}(t)$; i.e., $o_{rj} \leftarrow \{o_{rj} | o_{rj} \in O_{RJ}^{\min}(t)\}_{\max_URG_{rj}(t)}$. Otherwise, select the operation o_{rj} with the

- maximum $ET_{ij}^{\max}(t)$ value from the set $ETO_{RJ}(t)$; i.e., $o_{ij} \leftarrow \{o_{ij} | o_{ij} \in ETO_{RJ}(t)\}_{\max_{ET_{ij}^{\max}(t)}}$.
- 2) Operation selection rule 2
If $ATO_{RJ}(t) = \emptyset$, select the operation o_{ij} with the maximum $URG_{ij}(t)$ value from the set $\{o_{ij} | o_{ij} \in O_{idle}^{RJ}(t)\}_{\max_{URG_{ij}(t)}}$. Otherwise, select the operation o_{ij} with the maximum $AT_{ij}^{\max}(t)$ value from the set $ATO_{RJ}(t)$; i.e., $o_{ij} \leftarrow \{o_{ij} | o_{ij} \in ATO_{RJ}(t)\}_{\max_{AT_{ij}^{\max}(t)}}$.
- 3) Operation selection rule 3
If $O_{idle-f}^{RJ}(t) = \emptyset$, select the operation o_{ij} with the maximum $gap_{ij}(t)$ value from the set $O_{idle}^{RJ}(t)$; i.e., $o_{ij} \leftarrow \{o_{ij} | o_{ij} \in O_{idle}^{RJ}(t)\}_{\max_{gap_{ij}(t)}}$. Otherwise, select the operation o_{ij} with the maximum $gap_{ij}(t)$ value from the set $O_{idle-f}^{RJ}(t)$; i.e., $o_{ij} \leftarrow \{o_{ij} | o_{ij} \in O_{idle-f}^{RJ}(t)\}_{\max_{gap_{ij}(t)}}$.
- 4) Operation selection rule 4
If $O_{idle-f}^{RJ}(t) = \emptyset$, select the operation o_{ij} with the maximum $URG_{ij}(t)$ value from the set $O_{idle}^{RJ}(t)$; i.e., $o_{ij} \leftarrow \{o_{ij} | o_{ij} \in O_{idle}^{RJ}(t)\}_{\max_{URG_{ij}(t)}}$. Otherwise, select the operation o_{ij} with the maximum $URG_{ij}(t)$ value from the set $O_{idle-f}^{RJ}(t)$; i.e., $o_{ij} \leftarrow \{o_{ij} | o_{ij} \in O_{idle-f}^{RJ}(t)\}_{\max_{URG_{ij}(t)}}$.
- 5) Operation selection rule 5
If $O_{idle-f}^{RJ}(t) = \emptyset$, select the process o_{ij} with the minimum $Tard_{ij}^{\min}(t)$ value from the set $O_{idle}^{RJ}(t)$; i.e., $o_{ij} \leftarrow \{o_{ij} | o_{ij} \in O_{idle}^{RJ}(t)\}_{\min_{Tard_{ij}^{\min}(t)}}$. Otherwise, select the operation o_{ij} with the minimum $Tard_{ij}^{\min}(t)$ value from the set $O_{idle-f}^{RJ}(t)$; i.e., $o_{ij} \leftarrow \{o_{ij} | o_{ij} \in O_{idle-f}^{RJ}(t)\}_{\min_{Tard_{ij}^{\min}(t)}}$.
- 6) Operation selection rule 6
Select the process o_{ij} with the minimum $Tard_{ij}^{\min}(t)$ value from the set $O_{idle}^{RJ}(t)$; i.e., $o_{ij} \leftarrow \{o_{ij} | o_{ij} \in O_{idle}^{RJ}(t)\}_{\min_{Tard_{ij}^{\min}(t)}}$.
- 7) Operation selection rule 7
If $O_{idle-f}^{RJ}(t) = \emptyset$, select the process o_{ij} with the minimum $t_{ij}^{\min}(t)$ value from the set $O_{idle}^{RJ}(t)$; i.e., $o_{ij} \leftarrow \{o_{ij} | o_{ij} \in O_{idle}^{RJ}(t)\}_{\min_{t_{ij}^{\min}(t)}}$. Otherwise, select the operation o_{ij} with the minimum $t_{ij}^{\min}(t)$ value from the set $O_{idle-f}^{RJ}(t)$; i.e., $o_{ij} \leftarrow \{o_{ij} | o_{ij} \in O_{idle-f}^{RJ}(t)\}_{\min_{t_{ij}^{\min}(t)}}$.
- 8) Operation selection rule 8
select the process o_{ij} with the minimum $t_{ij}^{\min}(t)$ value from the set $O_{idle}^{RJ}(t)$; i.e., $o_{ij} \leftarrow \{o_{ij} | o_{ij} \in O_{idle}^{RJ}(t)\}_{\min_{t_{ij}^{\min}(t)}}$.
- 9) Operation selection rule 9
If $O_{idle-f}^{RJ}(t) = \emptyset$, select the process o_{ij} with the minimum $pe_{ij}^{\min}(t)$ value from the set $O_{idle}^{RJ}(t)$; i.e., $o_{ij} \leftarrow \{o_{ij} | o_{ij} \in O_{idle}^{RJ}(t)\}_{\min_{pe_{ij}^{\min}(t)}}$. Otherwise, select the operation o_{ij} with the minimum $pe_{ij}^{\min}(t)$ value from the set $O_{idle-f}^{RJ}(t)$; i.e., $o_{ij} \leftarrow \{o_{ij} | o_{ij} \in O_{idle-f}^{RJ}(t)\}_{\min_{pe_{ij}^{\min}(t)}}$.
- 10) Operation selection rule 10
Select the process o_{ij} with the minimum $pe_{ij}^{\min}(t)$ value from the set $O_{idle}^{RJ}(t)$; i.e., $o_{ij} \leftarrow \{o_{ij} | o_{ij} \in O_{idle}^{RJ}(t)\}_{\min_{pe_{ij}^{\min}(t)}}$.
- 11) Operation selection rule 11
If $O_{idle-f}^{RJ}(t) = \emptyset$, randomly select an operation o_{ij} from the set $O_{RJ}(t)$ using the Equation $o_{ij} \leftarrow \{o_{ij} | o_{ij} \in O_{idle}^{RJ}(t)\}_{\text{random}}$. Otherwise, randomly select an operation from the set $O_{idle-f}^{RJ}(t)$; i.e., $o_{ij} \leftarrow \{o_{ij} | o_{ij} \in O_{idle-f}^{RJ}(t)\}_{\text{random}}$.

12) Operation selection rule 12

Randomly select an operation o_{ij} from the set $O_{RJ}(t)$ using the Equation $o_{ij} \leftarrow \{o_{ij} | o_{ij} \in O_{idle}^{RJ}(t)\}_{\text{random}}$.

After selecting operation o_{ij} using the operation selection rules, due to the multiplicity characteristic of the MDFJSP, multiple jobs may be in the same operation stage o_{ij} . Therefore, we select the specific operation o_{mij} with the smallest due date from operation stage o_{ij} ; i.e., $o_{mij} \leftarrow \{o_{mij} | n \in I_{ij}(t)\}_{\min_{d_m}}$. This is to choose the job with the shortest due date from the job set $I_{ij}(t)$.

4.6. Machine assignment rules

This subsection defines ten machine assignment rules as the set of available actions for the machine policy network. At each decision point t , the machine policy network selects a machine assignment rule a_{mt} based on s_t and a_{ot} . Subsequently, a machine is selected to process the selected operation o_{mij} based on a_{mt} .

1) Machine assignment rule 1

If $M_{ij-f}^{\text{idle}}(t) = \emptyset$, select machine m from $M_{ij}^{\text{idle}}(t)$ to process the selected operation o_{mij} according to $m \leftarrow \{m | m \in M_{ij}^{\text{idle}}(t)\}_{\max_{gap_{mij}(t)}}$, with m having the maximum value of $gap_{mij}(t)$. Otherwise, select machine m from $M_{ij-f}^{\text{idle}}(t)$ to process the selected operation o_{mij} according to $m \leftarrow \{m | m \in M_{ij-f}^{\text{idle}}(t)\}_{\max_{gap_{mij}(t)}}$, with m having the maximum value of $gap_{mij}(t)$.

2) Machine assignment rule 2

If $M_{ij-f}^{\text{idle}}(t) = \emptyset$, select machine m from $M_{ij}^{\text{idle}}(t)$ to process the selected operation o_{mij} according to $m \leftarrow \{m | m \in M_{ij}^{\text{idle}}(t)\}_{\min_{t_{ijm}}}$. Otherwise, select machine m from $M_{ij-f}^{\text{idle}}(t)$ to process the selected operation o_{mij} according to $m \leftarrow \{m | m \in M_{ij-f}^{\text{idle}}(t)\}_{\min_{t_{ijm}}}$.

3) Machine assignment rule 3

Select machine m from $M_{ij}^{\text{idle}}(t)$ to process the selected operation o_{mij} according to $m \leftarrow \{m | m \in M_{ij}^{\text{idle}}(t)\}_{\min_{t_{ijm}}}$, with m having the shortest processing time among the selectable machines for operation o_{mij} .

4) Machine assignment rule 4

If $M_{ij-f}^{\text{idle}}(t) = \emptyset$, select machine m from $M_{ij}^{\text{idle}}(t)$ to process the selected operation o_{mij} according to $m \leftarrow \{m | m \in M_{ij}^{\text{idle}}(t)\}_{\min_{gap_m(t)}}$, with m having the maximum value of $gap_m(t)$. Otherwise, select machine m from $M_{ij-f}^{\text{idle}}(t)$ to process the selected operation o_{mij} according to $m \leftarrow \{m | m \in M_{ij-f}^{\text{idle}}(t)\}_{\max_{gap_m(t)}}$, with m having the maximum value of $gap_m(t)$.

5) Machine assignment rule 5

If $M_{ij-f}^{\text{idle}}(t) = \emptyset$, select machine m from $M_{ij}^{\text{idle}}(t)$ to process the selected operation o_{mij} according to $m \leftarrow \{m | m \in M_{ij}^{\text{idle}}(t)\}_{\min_{pe_{ijm}}}$. Otherwise, select machine m from $M_{ij-f}^{\text{idle}}(t)$ to process the selected operation o_{mij} according to $m \leftarrow \{m | m \in M_{ij-f}^{\text{idle}}(t)\}_{\min_{pe_{ijm}}}$.

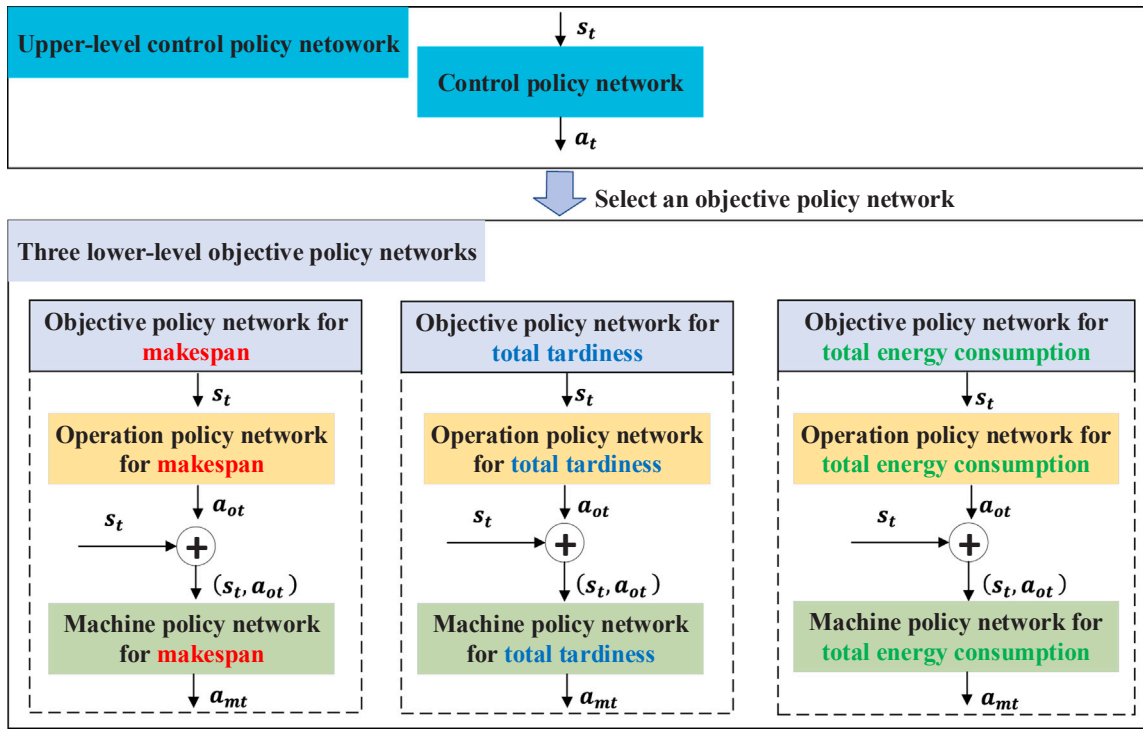


Fig. 3. The hierarchical multi-policy network framework.

6) Machine assignment rule 6

Select machine m from $M_{rj}^{idle}(t)$ to process the selected operation

$$o_{rj} \text{ according to } m \leftarrow \left\{ m | m \in M_{rj}^{idle}(t) \right\}_{\min -pe_{rjm}}.$$

7) Machine assignment rule 7

If $M_{rj-f}^{idle}(t) = \emptyset$, select machine m from $M_{rj}^{idle}(t)$ to process the selected operation o_{rj} according to $m \leftarrow \left\{ m | m \in M_{rj}^{idle}(t) \right\}_{\max -p_m^{idle}}$. Otherwise, select machine m from $M_{rj-f}^{idle}(t)$ to process the selected operation o_{rj} according to $m \leftarrow \left\{ m | m \in M_{rj-f}^{idle}(t) \right\}_{\max -p_m^{idle}}$.

8) Machine assignment rule 8

Select machine m from $M_{rj}^{idle}(t)$ to process the selected operation

$$o_{rj} \text{ according to } m \leftarrow \left\{ m | m \in M_{rj}^{idle}(t) \right\}_{\max -p_m^{idle}}.$$

9) Machine assignment rule 9

If $M_{rj-f}^{idle}(t) = \emptyset$, machine m randomly from $M_{rj}^{idle}(t)$ to process the selected operation o_{rj} according to $m \leftarrow \left\{ m | m \in M_{rj}^{idle}(t) \right\}_{\text{random}}$. Otherwise, select machine m randomly from $M_{rj-f}^{idle}(t)$ to process the selected operation o_{rj} according to $m \leftarrow \left\{ m | m \in M_{rj-f}^{idle}(t) \right\}_{\text{random}}$, with m having the maximum value of $gap_m(t)$.

10) Machine assignment rule 10

Select machine m randomly from $M_{rj}^{idle}(t)$ to process the selected operation o_{rj} according to $m \leftarrow \left\{ m | m \in M_{rj}^{idle}(t) \right\}_{\text{random}}$.

4.7. Reward function

In this section, the immediate rewards for three objective policy networks are defined as r_{IT} , r_{IC} , and r_{IE} . r_t is defined as the immediate

reward for the control policy network.

(1) The reward function for three objective policy networks

The immediate reward function of makespan objective policy network is defined based on the actual total tardiness of all jobs at two continuous decision points $T_a(t-1)$ and $T_a(t)$, as given in Eq. (46).

$$r_{IT} = T_a(t-1) - T_a(t) \quad (46)$$

$$R_T = \sum_{t=1}^T T_a(t-1) - T_a(t) = T_a(0) - T_a(T) = -T_{\max} \quad (47)$$

The cumulative reward R_T can be calculated with Eq. (47). $T_a(0)$ represents the actual total tardiness of jobs at the initial time, so $T_a(0) = 0$. $T_a(T)$ represents the actual total tardiness of jobs at the scheduling completion time, so $T_a(T) = T_{\max}$. Therefore, the cumulative reward is $R_T = -T_{\max}$. Minimizing the total tardiness $T_{\max}$ is equivalent to maximizing the cumulative reward R_T .

Similarly, the immediate reward r_{IC} for the makespan objective policy network is defined based on the maximum completion times of all machines at adjacent decision points, as follows in Eq. (48). $C(t)$ represents the maximum completion times of all machines, then $C(t) = \max\{CT_m(t) | m \in M_m\}$.

$$r_{IC} = C(t-1) - C(t) \quad (48)$$

The immediate reward r_{IE} of total energy consumption objective policy network is defined as Eq. (49). $E(t)$ represents the total energy consumption.

$$r_{IE} = E(t-1) - E(t) \quad (49)$$

(2) The reward function for control policy network

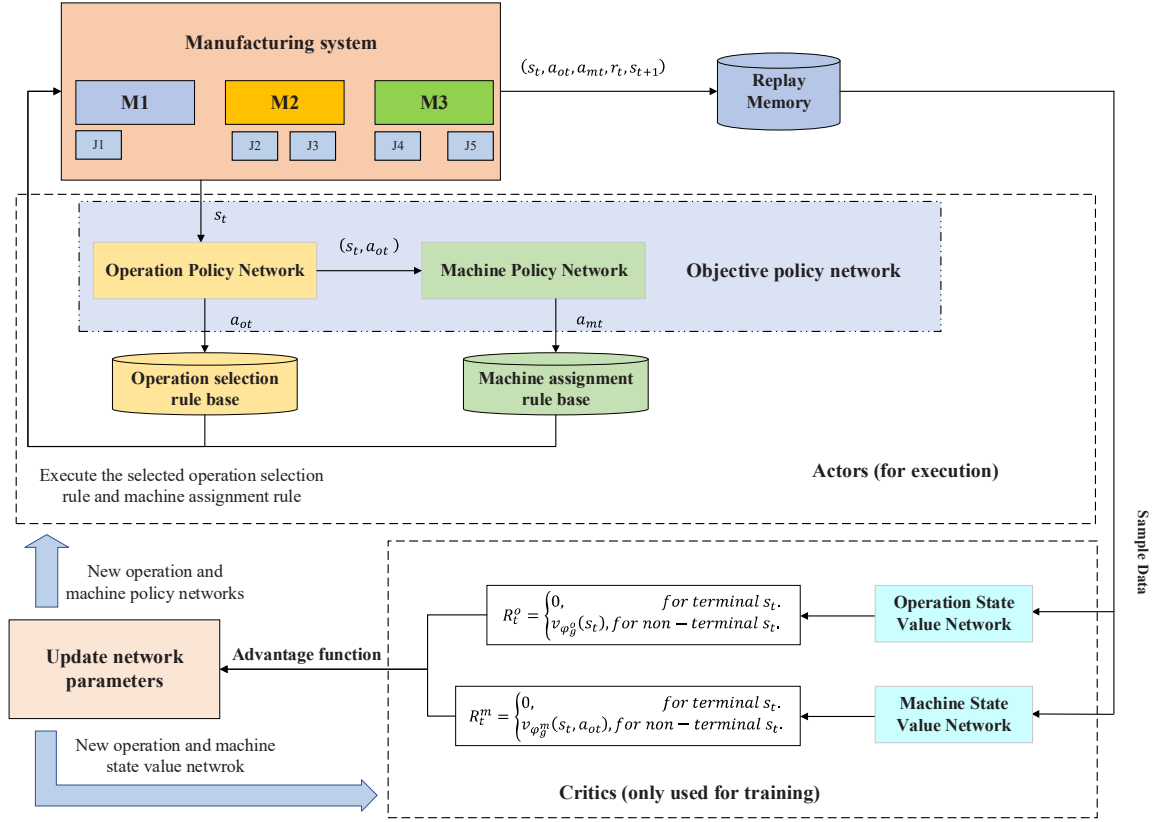


Fig. 4. The collaborative mechanism of different networks in each objective policy network training.

The r_t is defined based on r_{tT} , r_{tC} , and r_{tE} . To ensure robustness across various problem scales, the immediate reward r_t is scaled using the maximum metrics $C_{\max}$, $T_{\max}$, and $E_{\max}$ from the corresponding trained objectives policy networks for completion time, tardiness, and energy consumption.

$$r_t = \frac{r_{tT}}{T_{\max}} + \frac{r_{tC}}{C_{\max}} + \frac{r_{tE}}{E_{\max}} \quad (50)$$

4.8. Hierarchical multi-policy network framework

The proposed hierarchical multi-policy network framework is shown in Fig. 3, which integrates an upper-level control policy network and three lower-level objective policy networks. The upper-level control policy network is used for selecting a lower-level objective policy network according to action a_t . Three lower-level objective policy networks are designed for optimizing makespan, total tardiness time, and total energy consumption, respectively. The three objective networks are identical and separate networks that share the same network structure. Each objective policy network consists an operation policy network and a machine policy network, which are used to select operation selection rules and machine assignment rules, respectively.

The control policy network and the three objective policy networks share the same state features, as defined in Section 4.4. The control policy network utilizes the three objective policy networks as its actions. Each objective policy network executes action selection based on defined operational rules and machine assignment rules. Furthermore,

the three objective policy networks are assigned distinct reward functions: r_{tT} , r_{tC} , and r_{tE} , corresponding to different objectives. The control policy network uses r_t as its reward function. The specific definitions of these reward functions are provided in Section 4.7.

(1) Three lower-level objective policy networks

We design three objective networks for three different objectives. Additionally, three operation policy networks and three machine policy networks were developed, aligning with their respective objective networks. Both the operation and the machine policy networks ($\pi_{\theta_g^o}$ and $\pi_{\theta_g^m}$) are implemented as deep neural networks. The operation policy network features an input layer with thirty nodes, succeeded by hidden layers of three, each comprising two hundred nodes, and it concludes with an output layer of SoftMax with fifteen nodes. Similarly, the machine policy network comprises thirty-one nodes of the input layer, three hidden layers, each containing two hundred nodes, and an output layer of SoftMax with ten nodes. Policy network training relies on the state value network to estimate the cumulative expected reward of states, which serves as a critical guide for policy network optimization. Therefore, state value networks are constructed for operation and machine policy networks to ensure accurate and comprehensive evaluation of state values. The operation state value network $v_{\theta_g^o}$ includes an input layer with thirty nodes, followed by three hidden layers, each with two hundred nodes, and is finalized with a fully connected layer featuring one node. The machine state value network $v_{\theta_g^m}$ includes an input layer

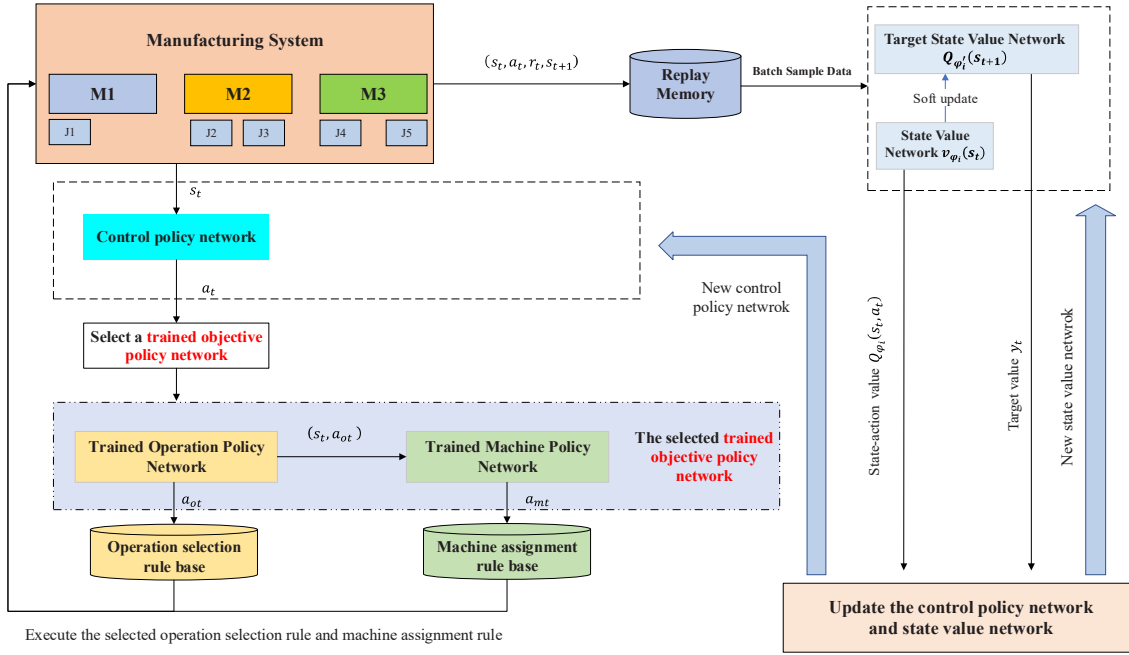


Fig. 5. The hierarchical collaborative mechanism of different networks in the control network training.

with thirty-one nodes, followed by three hidden layers, each with two hundred nodes, and is finalized with a fully connected layer featuring one node.

The three policy networks are trained individually using the network collaboration mechanism shown in Fig. 4. The operation selection rule base and the machine assignment rule base are constructed based on the defined operation selection rules and machine assignment rules, respectively. These rule bases serve as sets of actions for the operation policy network and the machine policy network, respectively. At each decision point t , the operation policy network selects an operation selection rule a_{ot} based on the state feature vector of the scheduling environment, and then the machine policy network selects a machine assignment rule a_{mt} based on the chosen operation selection rule a_{ot} and the current scheduling state s_t . Then, the two-dimensional discrete action $\mathbf{a}_t = (a_{ot}, a_{mt})$ is executed to assign a specific operation o_{mj} to an available machine m , completing the operation selection and machine assignment process at decision point t . The operation and machine state value networks estimate the value functions for the input states, facilitating the calculation of advantage functions, policy losses, and value losses. Subsequently, policy and value networks are updated via gradient descent, enabling iterative optimization of policy networks under the directive of specific objective strategy network reward functions.

(2) The upper-level control policy network

The upper-level control policy network features an input layer with thirty nodes, succeeded by hidden layers of three, each comprising two hundred nodes, and it concludes with an output layer of SoftMax with three nodes. The two state-action value networks $Q_{\theta_{g1}}$ and $Q_{\theta_{g2}}$ include an input layer with thirty nodes, followed by three hidden layers. Each hidden layer has two hundred nodes, and is finalized with an output layer featuring three nodes. Fig. 5 illustrates the hierarchical collaborative mechanism between the upper-level control policy network and the trained lower-level objective policy networks. The upper-level network acts as a controller, using the three trained objective

networks as discrete actions to select the appropriate objective based on the given state. Subsequently, the selected objective policy network serves as the executor at the lower level. It implements operation selection and machine assignment rules through its operation and machine policy networks according to the state, thereby achieving the specific optimization objective. This hierarchical collaboration facilitates effective interaction between the high and lower-level networks, providing a robust structural foundation for conducting complex intelligent decision-making tasks.

4.9. Two-stage training method

The two-stage training methodology involves the training of objective policy networks and control policy networks, as depicted in Fig. 6. Initially, each objective policy network is trained separately based on state features and scheduling rules using the Advantage Actor-Critic (A2C) algorithm. Each objective policy network is guided by its corresponding reward function, which ensures that the network converges effectively within the context of its specific objective. Upon the completion of this stage, the trained objective policy networks are used as potential actions for the subsequent training of the control policy network using the SAC algorithm. Decision points t , are defined as the moments of new order arrivals, machine breakdowns, and the completion of processing tasks by machines. The detailed algorithms for both the objective policy network training and control policy network training stages are delineated as follows.

(1) The training stage of objective policy network

The training process is repeated for L epochs. At each epoch, the algorithm is trained on new randomly generated training instances to improve the generalization of the objective policy network. The overall training algorithm based on A2C is presented in Algorithm 1.

Algorithm 1 The training method of objective policy network.

Initialize parameters:

Operation policy network $\pi_{\theta_g^o}$ and machine policy network $\pi_{\theta_g^m}$.

Operation state value network $v_{\varphi_g^o}$ and machine state value network $v_{\varphi_g^m}$.

Reply memory D_g of objective g .

```

1 for  $epoch = 1:L$  do
2   Initialize a new instance as the train environment.
3   Observe the environment to obtain the initialize state  $s_0$ .
4   for  $t = 0:T$  do ( $t$  is the decision point,  $T$  is the terminal time)
5     //New order  $s$  arrivals at time  $A_s$ .
6     if order  $s$  arrives at time  $A_s$  then
7       Add the jobs from the new order to the job set  $I_{rj}(t), \forall r, j$ .
8       Add the jobs from the new order to the job set  $I_{rj}^+(t), \forall r, j$ .
9       Resolve the fluid model and update its dynamic parameters.
10    end if
11    //The  $k - th$  breakdown of machine  $m$ .
12    if the  $kth$  breakdown of machine  $m$  occurs then
13      if there exists an operation  $o_{rnj}$  being processed on machine  $m$  then
14         $ct_{rnj} \leftarrow ct_{rnj} + bt_{mk}, CT_m(t) \leftarrow ct_{rnj}$ .
15      else
16         $CT_m(t) \leftarrow bs_{mk} + bt_{mk}$ .
17      end if
18      Resolve the fluid model and update its dynamic parameters.
19    end if
20    Observe the environment to obtain state  $s_t = [v_t, v_t - v_{t-1}]$ .
21    // Select action  $a_t$  based on policy networks.
22    Select an operation selection rule  $a_{ot}$  based on the operation policy network  $\pi_{\theta_g^o}(a_{ot}|s_t)$ .
23    Select a machine assignment rule  $a_{mt}$  based on the machine policy network  $\pi_{\theta_g^m}(a_{mt}|s_t, a_{ot})$ .
24    The selected two-dimensional discrete action is  $a_t = (a_{ot}, a_{mt})$ .
25    // Execute action  $a_t$ .
26    Select a job  $o_{rnj}$  based on  $a_{ot}$ , and assign the selected job  $o_{rnj}$  to a machine  $m$  based on  $a_{mt}$ .
27    Observe the environment to obtain the next state  $s_{t+1}$  and immediate reward  $r_t \leftarrow r_{tg}, g \in \{C, T, E\}$ .
28    Save the experience  $(s_t, a_{ot}, a_{mt}, r_t, s_{t+1})$  to  $D_g$ .
29  end for
30  //Sample trajectory data from  $D_g$ . Calculate the future discounted rewards, advantage function, and gradients.
31   $R_t^o = \begin{cases} 0, & \text{for terminal } s_t. \\ v_{\varphi_g^o}(s_t), & \text{for non-terminal } s_t. \end{cases}$ 
32   $R_t^m = \begin{cases} 0, & \text{for terminal } s_t. \\ v_{\varphi_g^m}(s_t, a_{ot}), & \text{for non-terminal } s_t. \end{cases}$ 
33  for  $t = 0:T$  do
34     $R_t^o \leftarrow r_t + \gamma R_{t+1}^o$ .
35     $A_t^o \leftarrow R_t^o - v_{\varphi_g^o}(s_t)$ .
36     $R_t^m \leftarrow r_t + \gamma R_{t+1}^m$ .
37     $A_t^m \leftarrow R_t^m - v_{\varphi_g^m}(s_t, a_{ot})$ .
38  end for
39  //Update the parameters of policy network.
40   $\theta_g^o \leftarrow \theta_g^o + \alpha_A \nabla_{\theta_g^o} T^{-1} \sum \left( -A_t^o \log \pi_{\theta_g^o}(a_{ot}|s_t) - \beta H(\pi_{\theta_g^o}(s_t)) \right)$ .
41   $\theta_g^m \leftarrow \theta_g^m + \alpha_A \nabla_{\theta_g^m} T^{-1} \sum \left( -A_t^m \log \pi_{\theta_g^m}(a_{mt}|s_t, a_{ot}) - \beta H(\pi_{\theta_g^m}(s_t, a_{ot})) \right)$ .
42  //Update the parameters of state value network.
43   $\varphi_g^o \leftarrow \varphi_g^o + \alpha_C \nabla_{\varphi_g^o} T^{-1} \sum \left( R_t^o - v_{\varphi_g^o}(s_t) \right)^2$ .
44   $\varphi_g^m \leftarrow \varphi_g^m + \alpha_C \nabla_{\varphi_g^m} T^{-1} \sum \left( R_t^m - v_{\varphi_g^m}(s_t, a_{ot}) \right)^2$ .
45 end for

```

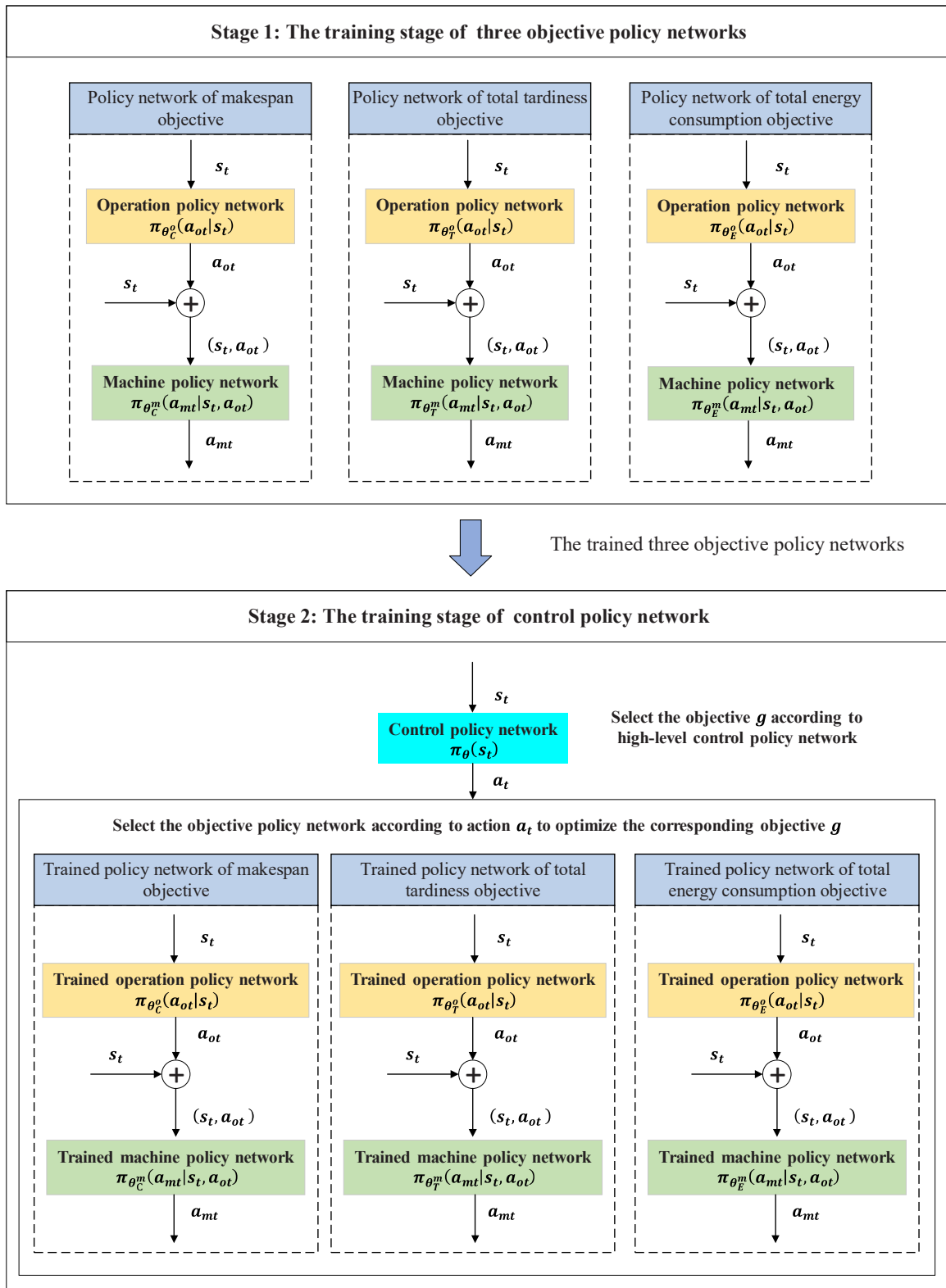


Fig. 6. The two-stage training method.

(2) The training stage of the control policy network

The training process is repeated for L epochs. At each epoch, the algorithm is trained on new randomly generated training instances to

improve the generalization of the control policy network. The overall training method based on SAC is presented in [Algorithm 2](#).

Algorithm 2. The training method of control policy network.

Initialize parameters:

Control policy network $\pi_\theta: S \rightarrow [0,1]^{|A|}$.

Two state-action value networks $Q_{\varphi_1}: S \rightarrow R^{|A|}$, $Q_{\varphi_2}: S \rightarrow R^{|A|}$.

Two target state-action value networks $Q_{\varphi'_1}: S \rightarrow R^{|A|}$, $Q_{\varphi'_2}: S \rightarrow R^{|A|}$.

Replay memory D .

46 **for** $epoch = 1:L$ **do**

47 Initialize a new instance as the train environment.

48 Observe the environment to obtain the initialize state s_0 .

49 **for** $t = 0:T$ **do** (t is the decision point, T is the terminal time)

50 //New order s arrivals at time A_s .

51 **if** *order s arrives* **then**

52 Add the jobs from the new order to the job set $I_{rj}(t), \forall r, j$.

53 Add the jobs from the new order to the job set $I_{rj}^+(t), \forall r, j$.

54 Resolve the fluid model and update its dynamic parameters.

55 **end if**

56 //The $k - th$ breakdown of machine m .

57 **if** *the kth breakdown of machine m occurs* **then**

58 **if** *there exists an operation o_{rnj} being processed on machine m* **then**

59 $ct_{rnj} \leftarrow ct_{rnj} + bt_{mk}, CT_m(t) \leftarrow ct_{rnj}$.

60 **else**

61 $CT_m(t) \leftarrow bs_{mk} + bt_{mk}$.

62 **end if**

63 **end if**

64 Observe the environment to obtain state $s_t = [v_t, v_t - v_{t-1}]$.

65 Select an operation selection rule $a_t \leftarrow \pi_\theta(s_t)$ based on the control policy network π_θ .

66 // Execute action a_t .

67 Select the objective $g \in \{C, T, E\}$ according to action a_t .

68 Select an operation selection rule a_{ot} based on the trained operation policy network $\pi_{\theta_g^o}(a_{ot}|s_t)$.

69 Select a machine assignment rule a_{mt} based on the trained machine policy network $\pi_{\theta_g^m}(a_{mt}|s_t, a_{ot})$.

70 The selected two-dimensional discrete action is $a_t = (a_{ot}, a_{mt})$.

71 Select a job o_{rnj} based on a_{ot} , and assign the selected job o_{rnj} to a machine m based on a_{mt} .

72 Obtain the next state s_{t+1} and immediate reward r_t .

73 Save the experience (s_t, a_t, r_t, s_{t+1}) to D .

74 Sample B batch (s_t, a_t, r_t, s_{t+1}) data from D .

75 // Calculate the target value.

76 $y_t = r_t + \gamma \pi_\theta(s_t)^T \left(\min_{i=1,2} Q_{\varphi'_i}(s_{t+1}) - \alpha \log(\pi_\theta(s_{t+1})) \right)$.

77 // Update the state-action value network.

78 $\nabla_{\varphi_i} J(\varphi_i) = B^{-1} \sum \nabla_{\varphi_i} (Q_{\varphi_i}(s_t, a_t) - y_t)^2, i \in \{1,2\}$.

79 // Update the control policy network.

80 $\nabla_\theta J(\theta) = B^{-1} \sum \nabla_\theta \left(\pi_\theta(s_t)^T \left(\alpha \log(\pi_\theta(s_t)) - \min_{i=1,2} Q_{\varphi_i}(s_t) \right) \right)$.

81 // Soft the target state-action value network.

82 $\varphi'_i \leftarrow \tau \varphi_i + (1 - \tau) \varphi'_i, i \in \{1,2\}$.

83 //Update entropy regularization coefficient.

84 $\nabla_\alpha J(\alpha) = \pi_\theta(s_t)^T \nabla_\alpha [-\alpha \log(\pi_\theta(s_t)) - \alpha H]$.

85 **end for**

86 **end for**

Table 6
Instance parameter settings.

Parameter	Value
Number of machines (M)	Randi [10,20]
Total number of job types (R)	Randi [5,15]
Total number of operations for each job type in an order (J_r)	Randi [5,10]
Number of available machines for each operation	Randi [1, M]
Processing time of each operation on the available machines (t_{rjm})	Randi [10,20]
Total number of newly arrived orders (S)	Randi [1,5]
Number of jobs for each job type in the newly arrived order (N_{sr})	Randi [5,10]
Time interval between the arrivals of new orders ($1/\lambda_{new}$)	Randi [100,200]
Urgency parameter of the due date (DDT)	Randf [0.5,1.5]
Processing power of operation on the available machines (p_{rjm})	Randi [10,200]
Machine idle power (p_m^{idle})	Randi [1,9]
Mean time between failure (λ_{MTBF})	190
Mean time to repair (λ_{MTTR})	10

Table 7
Training algorithm parameter settings.

Algorithm	Parameter	Value
A2C	Total training epochs (L)	1000
	Policy entropy coefficient (β)	0.001
	Discount factor (γ)	0.99
	Objective policy network learning rate (α_A)	0.0003
	State value network learning rate (α_C)	0.0003
	Optimizer	Adam
SAC	Total training epochs (L)	1000
	Sample batch size (B)	32
	Replay memory size	100000
	State-action value network learning rate	0.0003
	Control policy network learning rate	0.0003
	Discount factor (γ)	0.99
	Soft update factor (τ)	0.005
	Optimizer	Adam

5. Results and analyses

This section contains the parameter settings for the training and testing operations. The performance of the proposed HMPSAC algorithm is compared with existing well-known dispatching rules and deep reinforcement learning methods. The proposed HMPSAC and the comparative algorithms were implemented in Python 3.6 on a Windows 10 operating system with an Intel(R) Core (TM) I5-9300 CPU@2.40 GHz and 8 GB RAM. Each instance used in this paper is randomly generated based on the parameters in Table 6 which is generated according to Luo, et al. [34]. In Table 6, "Randi" and "Randf" denote uniform distributions for integers and real numbers.

In the training and testing environment, the order due date D_s is

defined as $D_s = A_s + DDT \bullet \left(\frac{\sum_{r=1}^R (N_{sr} \sum_{j=1}^{J_r} \bar{t}_{rj})}{2M} \right)$, where $\bar{t}_{rj}$ represents the

average processing time of operation o_{rj} on all selectable machines and DDT is the due date tightness. A smaller DDT value indicates a higher probability of tardiness for the jobs in the order.

The interval between two successive machine failures follows an exponential distribution with a mean of λ_{MTBF} , denoted as $\exp(1/\lambda_{MTBF})$. Additionally, the repair time for machine failures also adheres to an exponential distribution, expressed as $\exp(1/\lambda_{MTTR})$, where the mean $\lambda_{MTTR} = \bar{p}$, $\bar{p}$ is equal to the average processing time of all operations. It is defined that the ratio of the mean repair time to the sum of the mean repair time and the mean time between failures is 0.05, as captured by the expression $\frac{\lambda_{MTTR}}{\lambda_{MTTR} + \lambda_{MTBF}} = 0.05$ [54]. This relationship establishes a probabilistic framework for predicting machine breakdown and maintenance schedules in the operational environment.

5.1. Training process

During training, a new instance is created at random in each training epoch to increase the generalizability of the proposed HMPSAC algorithm in various production scenarios. The algorithm parameters used in the training process are shown in Table 7. In each training epoch, a newly generated training instance is used as the environment to train the policy network. The training environment, developed using Python, is available at https://github.com/Linshan-Ding/HMPSAC_Fluid_Mode1_MOMDFJSP/tree/main/environments. After each training epoch, the HMPSAC algorithm is tested on a generated validation instance, where the problem parameters M , S , and DDT are set to 15, 3, and 1.0, respectively. Fig. 7 shows that the three objective values of the test instance for each training epoch are based on the validation instance. The training curve indicates that the three targeted values decrease progressively throughout the training process and begin to stabilize after 400 cycles. This suggests that the HMPSAC algorithm is capable of effectively learning applicable scheduling strategies through its interaction with the shop scheduling environment.

5.2. Effectiveness verification for the learned policy

To verify the effectiveness of the upper-level control policy network in the proposed HMPSAC algorithm, this study conducted a series of comparative experiments. The HMPSAC algorithm was compared and analyzed against its variant methods: HMPSAC_C, HMPSAC_T, and HMPSAC_E. These variant algorithms are based on the three objective policy networks (maximum completion time, total tardiness time, and total energy consumption). Each variant algorithm directly outputs process selection and machine assignment strategies based on state

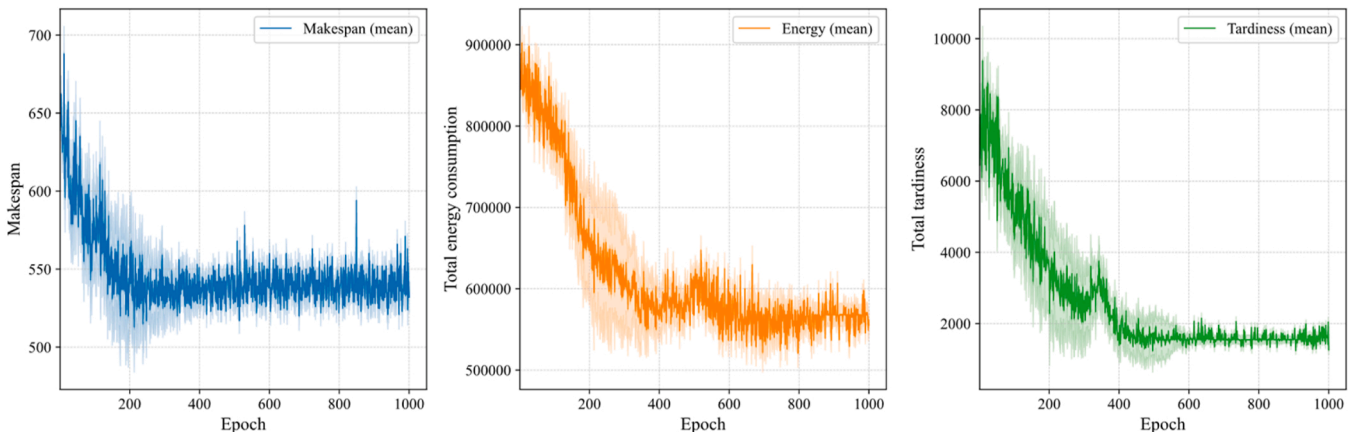


Fig. 7. The training curve of the HMPSAC algorithm.

Table 8

Comparison results of the HMPSAC algorithm and its variant methods.

DDT	M	S	HMPSAC		Random		HMPSAC_C		HMPSAC_T		HMPSAC_E	
			IGD	Std	IGD	Std	IGD	Std	IGD	Std	IGD	Std
0.5	10	1	0.010	0.006	0.707	0.010	0.240	0.015	0.088	0.043	0.367	0.022
		3	0.064	0.024	0.497	0.008	0.108	0.001	0.118	0.012	0.174	0.011
		5	0.011	0.019	1.062	0.098	0.104	0.008	0.063	0.014	0.309	0.047
	15	1	0.021	0.001	1.114	0.002	0.074	0.001	0.174	0.020	0.102	0.010
		3	0.075	0.011	1.311	0.008	0.100	0.009	0.151	0.003	0.097	0.009
		5	0.027	0.008	0.754	0.012	0.049	0.001	0.033	0.004	0.089	0.002
	20	1	0.016	0.003	0.523	0.002	0.107	0.002	0.084	0.009	0.089	0.002
		3	0.001	0.006	0.349	0.005	0.069	0.005	0.050	0.003	0.051	0.003
		5	0.127	0.011	3.362	0.000	0.190	0.000	0.225	0.002	0.152	0.003
	10	1	0.006	0.053	1.107	0.089	0.225	0.034	0.056	0.012	0.288	0.035
		3	0.039	0.016	0.576	0.017	0.125	0.003	0.018	0.003	0.307	0.011
		5	0.037	0.004	0.536	0.011	0.123	0.013	0.150	0.017	0.231	0.016
	15	1	0.050	0.023	26.980	0.000	0.121	0.000	0.249	0.001	0.149	0.000
		3	0.045	0.004	1.974	0.010	0.123	0.006	0.083	0.008	0.139	0.004
		5	0.030	0.006	1.387	0.003	0.083	0.005	0.105	0.001	0.113	0.004
	20	1	0.045	0.227	21.823	0.050	0.143	0.006	0.482	0.049	0.016	0.006
		3	0.065	0.095	4.140	0.015	0.255	0.006	0.207	0.010	0.220	0.006
		5	0.052	0.083	4.723	0.038	0.201	0.025	0.275	0.030	0.230	0.035
1.5	10	1	0.020	0.017	0.324	0.000	0.016	0.001	0.048	0.001	0.013	0.002
		3	0.047	0.021	6.708	0.029	0.162	0.004	0.141	0.020	0.280	0.004
		5	0.090	0.010	0.491	0.001	0.126	0.003	0.159	0.005	0.192	0.004
	15	1	0.027	0.036	0.673	0.000	0.024	0.000	0.128	0.000	0.037	0.000
		3	0.048	0.083	23.953	0.053	0.050	0.004	0.106	0.006	0.306	0.012
		5	0.053	0.168	4.972	0.008	0.186	0.011	0.201	0.011	0.424	0.003
	20	1	0.021	0.033	0.725	0.005	0.105	0.006	0.070	0.003	0.062	0.008
		3	0.040	0.016	1.426	0.004	0.057	0.002	0.086	0.002	0.058	0.001
		5	0.042	0.268	10.500	0.013	0.250	0.007	0.286	0.004	0.277	0.003
	mean		0.041	0.046	4.544	0.018	0.127	0.007	0.142	0.011	0.177	0.010

features. Furthermore, to demonstrate whether the lower-level objective strategy networks in the HMPSAC algorithm have learned effective scheduling rule selection strategies, the HMPSAC_C, HMPSAC_T, and HMPSAC_E algorithms were compared with a random action selection strategy (Random). In the Random strategy, both the process selection rules and machine assignment rules at decision points are generated through random selection, without guidance from a learning process. Our evaluation employed inverse generational distance (IGD)[55] which is a comprehensive metric evaluation of both the convergence and diversity for Pareto front solutions. Each method was independently run 30 times for each instance. The results of comparing the IGD and standard deviation (Std) of different methods in different instances are shown in Table 8.

The experimental results indicate that the HMPSAC algorithm obtained smaller IGD values in most of the test instances and also surpassed the comparative algorithms in terms of average performance across all instances, with an average IGD value of 0.041. This outcome demonstrates that the upper-level control policy network of the HMPSAC algorithm has successfully learned and implemented effective goal selection strategies. Furthermore, when compared to the random selection strategy (Random), all three variant algorithms (HMPSAC_C,

HMPSAC_T, and HMPSAC_E) achieve significantly lower IGD values, confirming the learning effectiveness of HMPSAC's objective policy network in the selection of scheduling rules. In addition, Table 8 shows that the Std values for the three variant algorithms are lower than HMPSAC. This is because HMPSAC considers three objectives simultaneously, which introduces a higher level of complexity and variability into the optimization process. Nevertheless, HMPSAC shows superior overall performance by effectively balancing the multiple objectives.

To further visualize the differences between the various methods, a raincloud plot was created based on all test cases, as shown in Fig. 8. This figure displays the distribution of IGD metric values for the HMPSAC algorithm and its variant algorithms across all test instances, including statistical information such as median and mean values. It is evident that the median and mean IGD values for the HMPSAC algorithm are significantly lower than those of the three variant algorithms across all instances. This indicates that the upper-level control policy network of HMPSAC has learned effective goal selection strategies, enabling it to choose the best temporary optimization goal at each decision point.

In addition, to further analyze the performance of HMPSAC and its variant algorithms for the three objectives, we recorded the mean values

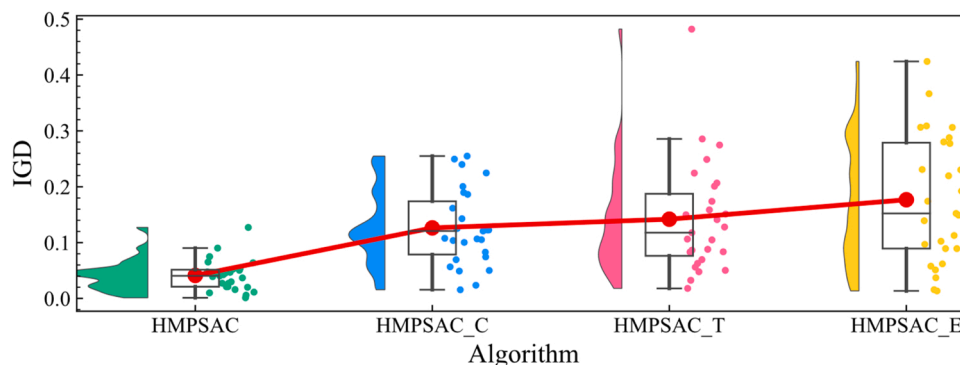


Fig. 8. The raincloud plot of IGD for all instances.

Table 9
Comparison results of the HMPSAC algorithm and its variant methods (mean).

DDT	M	S	HMPSAC			HMPSAC_C			HMPSAC_T			HMPSAC_E		
			Makespan	Tardiness	Energy	Makespan	Tardiness	Energy	Makespan	Tardiness	Energy	Makespan	Tardiness	Energy
0.5	10	1	451.47	7302.00	352630.53	433.30	7370.47		330453.10					
		3	1104.30	88349.10	944029.03	1191.03	85815.17		928207.23					
		5	2147.37	302677.60	1967296.13	2113.83	297441.90		1721101.13					
	15	1	229.77	1825.73	216321.27	227.40	1716.37		203374.43					
		3	600.93	15860.20	862518.03	642.43	13576.17		808892.10					
		5	793.93	25442.50	1158203.27	801.13	24958.77		1061911.33					
	20	1	209.90	2706.67	305948.43	238.37	2182.07		288738.67					
		3	613.33	5866.50	391463.73	599.37	5648.00		334177.23					
		5	775.43	8706.57	1140872.63	784.47	7255.80		1066888.60					
	1.0	1	480.00	905.70	388475.20	473.47	1062.90		373211.77					
		3	1195.53	32575.10	1032986.53	1314.73	32052.67		963904.33					
		5	2443.47	231072.70	1826685.80	2324.43	211865.40		1793935.23					
	15	1	327.33	449.37	445877.87	333.20	297.00		417304.87					
		3	615.93	4489.63	540810.33	617.07	4611.47		489733.93					
		5	975.87	32170.23	1469400.23	1042.90	30378.43		1404996.33					
	20	1	170.80	4.47	291433.13	176.47	0.63		277328.70					
		3	484.57	2346.57	602283.03	494.53	1994.20		561575.80					
		5	1270.00	23268.33	1461618.47	1266.70	21236.27		1321591.73					
	1.5	1	472.63	0.00	412195.93	474.30	0.00		402845.27					
		3	1365.40	22941.70	1049203.10	1322.77	18939.33		963933.83					
		5	1311.17	42950.87	1143204.33	1512.23	35697.80		1068726.10					
	15	1	211.87	65.23	192144.13	226.43	67.43		189809.77					
		3	675.13	328.63	973955.20	688.97	113.80		942720.47					
		5	894.83	392.67	1195581.33	914.87	97.07		1110060.23					
	20	1	184.13	0.00	297238.90	187.70	0.00		284124.80					
		3	465.93	105.20	785333.37	479.07	39.73		739228.70					
		5	814.17	62.37	1429888.97	830.63	18.63		1357090.70					

of all solutions obtained by HMPSAC for the three objectives, as well as the mean values of the solutions obtained by the three variant algorithms under their respective objectives. The results presented in Table 9 show that although the HMPSAC algorithm underperforms on individual objectives compared to the variant algorithms specifically designed for those objectives, Table 8 shows that HMPSAC achieves superior overall performance across all three objectives. This further validates the effectiveness and necessity of the proposed hierarchical multi-policy framework.

5.3. Comparison with dispatching rules

To verify the efficiency of the proposed HMPSAC, the HMPSAC algorithm is compared with nine well-known dispatching rules, namely, first-in-first-out (FIFO), earliest due date (EDD), maximum remaining processing time (MRT), critical ratio (CR), shortest processing time (SPT), CR combined with SPT (CR+SPT), FIFO combined with SPT (FIFO+SPT), MRT combined with SPT (MRT+SPT), and MRT combined with FIFO (MRT+FIFO) [56–60]. For each instance, each method is

Table 10
Comparison Results of HMPSAC and Dispatching Rules (IGD).

DDT	M	S	HMPSAC	FIFO	EDD	MRT	SPT	CR	CR+SPT	EDD+SPT	MRT+SPT	MRT+FIFO
0.5	10	1	0.00	2.12	1.77	2.08	1.83	1.78	1.77	1.81	2.02	2.00
		3	0.00	2.98	1.61	2.72	2.01	1.84	1.83	1.73	2.72	2.73
		5	0.00	4.25	2.39	5.35	4.17	2.78	2.84	3.05	5.37	5.27
	15	1	0.00	3.25	2.26	4.13	3.11	3.01	3.04	2.99	4.16	4.20
		3	0.00	6.46	4.06	8.67	4.41	4.46	4.43	4.33	8.80	8.69
		5	0.00	3.28	1.97	3.02	2.80	2.30	2.33	2.09	3.05	3.04
	20	1	0.00	1.77	1.57	1.78	1.79	1.42	1.41	1.91	1.88	1.85
		3	0.00	0.92	1.24	1.43	1.49	1.00	1.02	1.17	1.30	1.34
		5	0.00	16.76	10.35	18.64	13.12	10.77	11.02	10.87	18.62	18.65
	1.0	1	0.00	4.12	2.92	5.68	3.18	3.43	3.24	3.18	5.59	5.55
		3	0.00	2.62	1.37	3.19	2.00	1.43	1.47	1.36	3.21	3.21
		5	0.00	2.94	1.56	3.01	1.89	1.84	1.86	1.66	2.98	2.97
	15	1	0.00	241.10	141.67	255.66	228.89	150.57	154.84	218.72	242.62	253.33
		3	0.00	10.23	5.57	15.06	7.88	6.67	6.44	6.40	15.29	15.58
		5	0.00	7.16	3.97	7.74	4.74	4.21	4.21	4.00	7.80	7.75
	20	1	0.00	125.10	98.30	150.14	117.44	87.36	83.68	112.51	151.05	157.87
		3	0.00	18.54	10.42	19.85	12.73	11.80	11.75	11.23	19.52	19.83
		5	0.00	23.89	13.64	30.00	18.98	15.29	15.52	14.92	30.35	30.08
	1.5	1	0.00	3.86E+ 11	2.40E+ 11	4.52E+ 11	3.96E+ 11	3.51E+ 11	3.25E+ 11	3.67E+ 11	4.24E+ 11	4.40E+ 11
		3	0.00	106.94	62.66	188.26	88.00	70.91	67.40	60.78	188.77	193.39
		5	0.00	3.45	1.70	3.61	2.56	1.98	2.01	1.88	3.59	3.58
	15	1	0.00	4.01E+ 11	2.55E+ 11	1.70E+ 11	2.94E+ 11	2.75E+ 11	2.86E+ 11	3.12E+ 11	1.66E+ 11	2.08E+ 11
		3	0.00	167.25	110.09	282.73	149.96	95.44	90.33	99.13	283.92	282.31
		5	0.00	24.56	14.65	39.00	21.07	14.64	15.23	15.22	39.00	38.87
	20	1	0.00	3.28E+ 11	2.26E+ 11	1.92E+ 11	2.75E+ 11	2.66E+ 11	2.58E+ 11	2.77E+ 11	2.21E+ 11	1.57E+ 11
		3	0.00	11.50	9.33	27.44	16.16	12.11	12.25	10.56	28.94	27.79
		5	0.00	77.62	43.77	96.25	68.61	47.85	47.48	47.64	96.82	95.87
	Mean		0.00	4.13E+ 10	2.67E+ 10	3.01E+ 10	3.57E+ 10	3.30E+ 10	3.22E+ 10	3.54E+ 10	3.00E+ 10	2.98E+ 10

Table 11
Comparison Results of HMPSAC and Dispatching Rules (std).

DDT	M	S	HMPSAC	FIFO	EDD	MRT	SPT	CR	CR+SPT	EDD+SPT	MRT+SPT	MRT+FIFO
0.5	10	1	0.00	0.589	0.414	0.584	0.421	0.386	0.403	0.422	0.573	0.583
		3	0.00	0.116	0.058	0.115	0.079	0.062	0.061	0.063	0.115	0.115
		5	0.00	0.144	0.084	0.185	0.154	0.092	0.094	0.104	0.185	0.181
		15	1	0.00	0.598	0.333	0.818	0.437	0.453	0.450	0.820	0.817
			3	0.00	0.668	0.418	0.902	0.438	0.444	0.436	0.900	0.902
	20	5	0.00	0.104	0.038	0.106	0.060	0.042	0.044	0.042	0.106	0.106
		1	0.00	0.063	0.052	0.077	0.064	0.029	0.028	0.076	0.078	0.083
		3	0.00	0.080	0.092	0.100	0.102	0.083	0.088	0.093	0.100	0.102
		5	0.00	0.740	0.452	0.817	0.567	0.472	0.470	0.469	0.819	0.815
			0.00	0.220	0.143	0.331	0.149	0.169	0.167	0.156	0.304	0.327
1.0	10	1	0.00	0.111	0.040	0.150	0.065	0.048	0.047	0.045	0.149	0.148
		3	0.00	0.350	0.154	0.369	0.210	0.175	0.174	0.168	0.368	0.366
		15	1	0.00	104.148	60.538	107.193	103.611	64.483	66.864	101.062	106.009
			3	0.00	0.499	0.304	0.654	0.413	0.336	0.327	0.720	0.785
		5	0.00	1.050	0.580	1.159	0.698	0.602	0.597	0.582	1.146	1.151
	20	1	0.00	5.186	3.920	6.180	4.299	3.632	3.565	4.417	6.372	6.447
		3	0.00	4.931	2.738	5.249	3.413	3.043	3.060	2.958	5.209	5.258
		5	0.00	12.433	7.224	15.533	9.893	7.830	7.718	7.690	15.533	15.479
			0.00	1.15E+ 11	4.17E+ 10	4.15E+ 10	5.45E+ 10	2.07E+ 10	5.76E+ 10	1.66E+ 10	4.44E+ 10	3.48E+ 10
		3	0.00	11.637	6.748	20.426	9.723	7.434	7.166	6.839	20.561	20.403
1.5	10	5	0.00	0.221	0.081	0.241	0.138	0.094	0.090	0.095	0.241	0.242
		15	1	0.00	2.01E+ 11	1.28E+ 11	8.47E+ 10	1.47E+ 11	1.38E+ 11	1.43E+ 11	8.28E+ 10	1.04E+ 11
			3	0.00	268.137	155.014	447.468	223.010	143.161	144.434	440.524	437.999
		5	0.00	40.119	24.206	64.136	33.844	23.875	24.279	24.773	63.745	63.861
		20	1	0.00	2.58E+ 10	9.55E+ 09	3.78E+ 10	1.21E+ 10	4.13E+ 10	4.15E+ 09	2.39E+ 10	3.31E+ 10
			3	0.00	1.512	1.361	4.057	2.413	1.802	1.658	3.994	3.981
	20	5	0.00	109.276	60.072	135.206	97.584	66.195	67.086	67.885	134.961	135.675
		Mean	0.00	1.27E+ 10	6.64E+ 09	6.07E+ 09	7.91E+ 09	7.41E+ 09	9.45E+ 09	6.55E+ 09	5.60E+ 09	6.37E+ 09

independently run 30 times. Table 10 and Table 11 show the IGD and std of the experiments.

The experimental outcomes demonstrate that HMPSAC algorithm achieved the smallest IGD values in all 27 instances when compared to other well-known scheduling rules, indicating a significant performance advantage in complex multi-objective dynamic scheduling problems. Moreover, Table 11 shows that HMPSAC consistently achieves lower Std values across all instances, highlighting its superior stability and consistency compared to the nine well-known dispatching rules.

Fig. 9 shows the average decision times over 27 instances of different sizes, comparing the HMPSAC algorithm with other well-known dispatching rules. The computation times for HMPSAC and the other well-known dispatching rules are almost identical, with decisions being made in 0.1 seconds. However, it is noteworthy that the HMPSAC algorithm

has slightly longer computation time due to the additional time required to perform the following tasks: (1) Constructing state features: The HMPSAC algorithm constructs state features through dynamic fluid models and feature extraction processes, which requires some additional computation time. (2) Computation of Outputs for Control Policy Network and Goal Strategy Network: The HMPSAC algorithm uses two levels of policy networks, namely the control policy network and the objective policy network, to make decisions. This involves forward propagation of neural networks, which consumes computational time. Consequently, the average decision time for HMPSAC is slightly higher than other methods in most cases. However, this additional computation time is offset by improved scheduling performance because the HMPSAC algorithm is able to select the optimal objective strategy network and scheduling rules based on different state features.

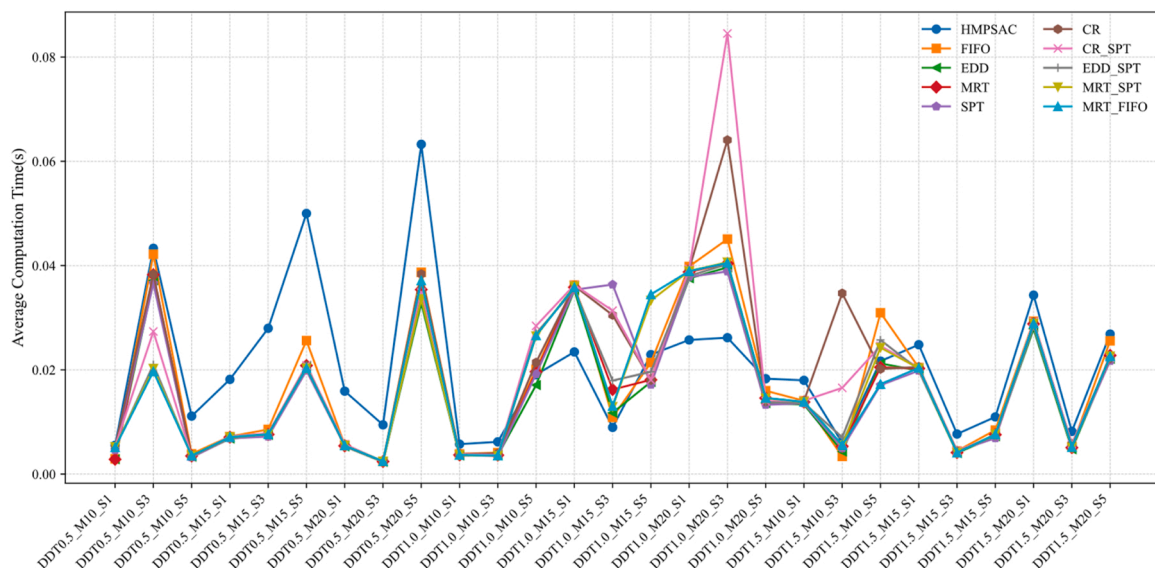


Fig. 9. The average decision-making time for HMPSAC and other renowned dispatching rules.

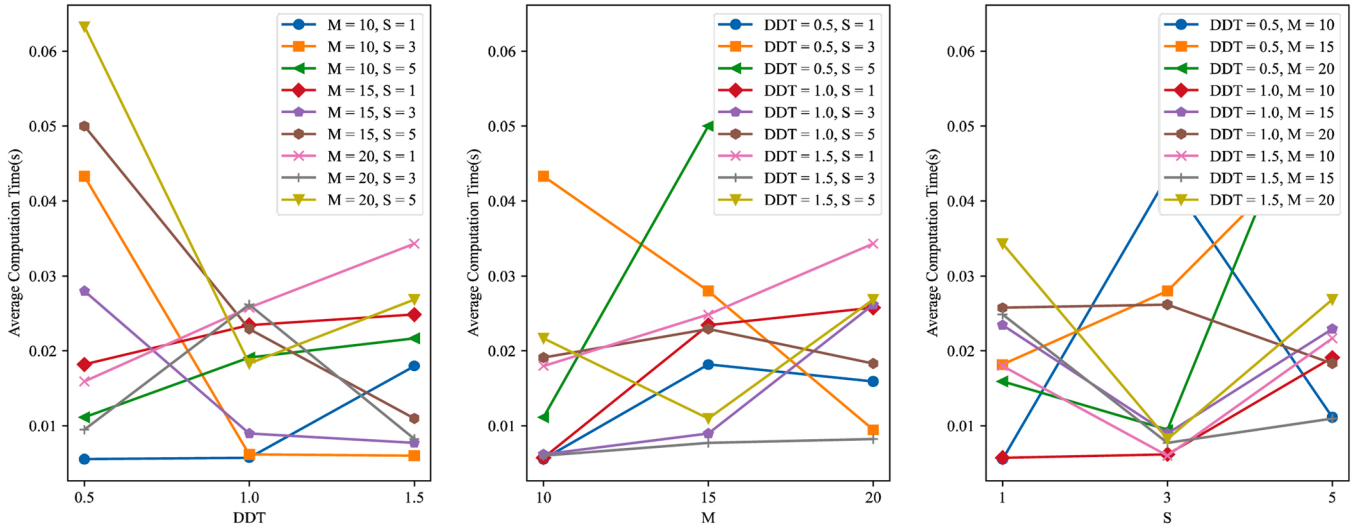


Fig. 10. The decision-making time variation curve for HMPsAC at different parameter levels.

Fig. 10 presents the variation curves of HMPsAC's average decision time at different parameter levels, comprising due date tightness (DDT), the total number of machines (M), and the total number of new order arrivals (S) on the average computational time at decision points (average decision time) for the HMPsAC algorithm. The results reveal that when S is small, DDT has a limited impact on the average decision time for the HMPsAC algorithm. As S increases, the average decision time tends to decrease with an increase in DDT, and the rise in the total number of machines M slightly elevates the algorithm's average decision time. Moreover, when DDT is small, the average decision time shows an increasing trend with the increase in S , while at higher levels of DDT, the influence of S on the average decision time is limited. This phenomenon can be attributed to the fact that a smaller DDT and a larger S lead to an increase in the total number of workpieces that the HMPsAC algorithm needs to consider at each decision point, which in turn increases the time required for the computation of state features, thereby increasing the average decision time. Furthermore, an increase in the total number of machines M expands the range of machines available for selection at each decision point, which in turn increases the computational load of the machine selection step and thus marginally extends the average decision time.

5.4. Comparison with DRL-based algorithms

To further validate the superiority of the proposed hierarchical multi-policy deep reinforcement learning framework of HMPsAC, a comparative analysis was conducted against four other multi-objective optimization methods based on deep reinforcement learning. These methods include the Hybrid Double Deep Q-Network (HDQN) [26], Dynamic Multi-objective Double Deep Q-Network (DMDDQN) [37], Two-Hierarchy Deep Q-Network (THDQN) [35], and Hierarchy Multi-Agent Proximal Policy Optimization (HMAPPO) [34]. Our evaluation employed generational distance (GD) [61], inverse generational distance (IGD) [55], and Spread [62] as metrics. GD measures the average distance between solutions in an algorithm's Pareto front and the nearest solutions in the true Pareto front, with lower values indicating better performance in terms of proximity to the true front. Spread quantifies how well an algorithm explores the entire Pareto front, with lower values indicating a more diverse approximation of the front.

For each instance, each method is independently run 30 times. Table 12 presents the outcomes of the experiments. The results indicate that the HMPsAC framework, which uses a hierarchical multi-policy network structure, outperforms other deep reinforcement learning methods such as HDQN, THDQN, HMAPPO, and DMDDQN in terms of

the Generational Distance (GD), Inverted Generational Distance (IGD), and Spread metrics. In addition, HMPsAC also achieved better overall average values for GD, IGD, and Spread across 27 instances compared to the other four deep reinforcement learning methods, further confirming the necessity and effectiveness of implementing a hierarchical multi-policy network structure.

Fig. 11 presents the raincloud plots for the GD, IGD, and Spread metrics of the HMPsAC and other comparative algorithms across all instances. As can be discerned from Fig. 11, the HMPsAC algorithm consistently achieves significantly smaller values in the GD, IGD, and Spread metrics for instances of varying sizes. Furthermore, the data points for GD and IGD are densely clustered around smaller values, indicating that HMPsAC performs stably when solving MOMDFJSP problems of different scales.

To visually demonstrate the performance of all algorithms in solving the MOMDFJSP, Fig. 12 displays the Pareto frontiers obtained by all comparative algorithms for instances of varying sizes. It can be observed that HMPsAC is capable of obtaining Pareto frontiers with good convergence for problems of different sizes. This indicates that the HMPsAC algorithm excels in addressing problems of varying scales and complexities, exhibiting strong universality and adaptability.

To visualize the distribution of Pareto solutions obtained by different algorithms across various dimensions, Fig. 13 respectively presents the marginal combination plots for three instances under different dimensions obtained by the comparative algorithms. We can clearly observe that the Pareto solutions obtained by the HMPsAC algorithm are more uniformly distributed across different objective dimensions, and the peaks of these distributions correspond to smaller objective values. This indicates that HMPsAC has achieved well-converged and evenly distributed Pareto frontiers across different objective dimensions, which further validates the previous analysis and conclusions.

In summary, the HMPsAC algorithm demonstrates exceptional performance in addressing the MOMDFJSP, exhibiting significant performance advantages. These benefits are attributed to several key features of HMPsAC: (1) Composite Reward Function: the HMPsAC algorithm employs a composite reward function based on multiple objectives, integrating several performance metrics. This design of the composite reward function facilitates a good trade-off and balance between different objectives, guiding the algorithm toward the global Pareto frontier. It shifts the focus from maximizing a single objective to enhancing convergence and search efficiency across multiple objectives. (2) Maximum Entropy Principle: The upper-level control strategy network of HMPsAC uses a maximum entropy reinforcement learning algorithm. This means the algorithm maximizes not just the reward

Table 12
Comparison results of HMPSAC and other deep reinforcement learning algorithms.

DDT	M	S	Spread					GD					IGD					
			HMPSAC	HMAPPO	DMDDQN	THDQN	HDQN	HMPSAC	HMAPPO	DMDDQN	THDQN	HDQN	HMPSAC	HMAPPO	DMDDQN	THDQN	HDQN	
0.5	10	1	0.261	0.688	0.799	0.756	0.731	0.000	0.031	0.031	0.071	0.062	0.000	0.120	0.217	0.174	0.170	
		3	0.869	0.878	0.907	0.843	0.852	0.004	0.009	0.003	0.000	0.004	0.064	0.091	0.158	0.116	0.117	
		5	0.566	0.562	0.625	0.773	0.744	0.003	0.000	0.010	0.115	0.072	0.014	0.071	0.061	0.271	0.234	
	15	1	0.335	0.718	0.647	0.783	0.801	0.000	0.086	0.031	0.161	0.179	0.000	0.262	0.092	0.478	0.540	
		3	0.630	0.850	0.892	0.892	0.943	0.000	0.035	0.035	0.063	0.159	0.000	0.092	0.111	0.173	0.404	
		5	0.736	0.764	0.702	0.846	0.874	0.005	0.002	0.008	0.071	0.116	0.020	0.033	0.027	0.218	0.296	
	20	1	0.416	0.804	0.790	0.903	0.900	0.000	0.061	0.053	0.194	0.205	0.000	0.194	0.182	0.512	0.558	
		3	0.497	0.904	0.925	0.917	0.925	0.000	0.080	0.107	0.103	0.101	0.000	0.265	0.347	0.301	0.304	
		5	0.732	0.683	0.770	0.900	0.907	0.000	0.000	0.039	0.231	0.270	0.084	0.137	0.133	0.556	0.692	
	1.0	10	1	0.413	0.674	0.834	0.904	0.887	0.000	0.017	0.074	0.252	0.169	0.001	0.093	0.316	0.749	0.570
			3	0.500	0.615	0.634	0.709	0.761	0.003	0.000	0.027	0.054	0.075	0.024	0.035	0.091	0.174	0.259
			5	0.888	0.893	0.917	0.881	0.883	0.003	0.009	0.001	0.000	0.003	0.072	0.169	0.144	0.224	0.211
15		1	1.034	1.011	1.019	1.004	1.003	0.000	0.019	0.044	0.475	0.798	0.000	0.356	0.142	1.459	1.792	
		3	0.828	0.867	0.885	0.945	0.953	0.000	0.028	0.026	0.140	0.232	0.011	0.150	0.149	0.525	0.606	
		5	0.527	0.502	0.598	0.578	0.602	0.000	0.000	0.005	0.007	0.039	0.029	0.079	0.111	0.112	0.114	
20		1	0.587	0.841	0.817	0.962	0.970	0.000	0.085	0.034	0.669	0.866	0.000	0.217	0.126	1.943	2.349	
		3	0.580	0.653	0.745	0.704	0.780	0.000	0.024	0.029	0.063	0.119	0.028	0.083	0.139	0.176	0.371	
		5	0.816	0.840	0.850	0.896	0.898	0.023	0.000	0.020	0.076	0.085	0.049	0.074	0.212	0.420	0.427	
1.5		10	1	0.882	0.786	0.855	0.766	0.754	0.021	0.012	0.000	0.003	0.000	0.082	0.032	0.068	0.065	0.042
			3	0.670	0.809	0.615	0.827	0.813	0.008	0.005	0.007	0.060	0.081	0.041	0.197	0.113	0.342	0.294
			5	0.891	0.888	0.901	0.842	0.845	0.000	0.022	0.000	0.000	0.005	0.098	0.197	0.149	0.293	0.286
	15	1	0.834	0.946	0.899	0.962	0.955	0.004	0.049	0.000	0.089	0.093	0.009	0.134	0.030	0.230	0.187	
		3	0.841	0.665	0.680	0.881	0.895	0.000	0.008	0.008	0.130	0.123	0.031	0.117	0.050	0.534	0.643	
		5	1.049	1.028	1.027	1.022	1.022	0.000	0.004	0.013	0.086	0.080	0.019	0.162	0.108	0.309	0.356	
	20	1	0.427	0.848	0.857	0.937	0.942	0.000	0.041	0.046	0.144	0.165	0.000	0.116	0.117	0.323	0.357	
		3	0.893	0.855	0.896	0.952	0.926	0.000	0.034	0.049	0.198	0.108	0.036	0.169	0.125	0.497	0.307	
		5	0.944	0.951	0.968	0.962	0.961	0.000	0.026	0.056	0.060	0.059	0.029	0.113	0.496	0.319	0.243	
	mean			0.691	0.797	0.817	0.865	0.871	0.003	0.025	0.028	0.130	0.158	0.028	0.139	0.149	0.426	0.471

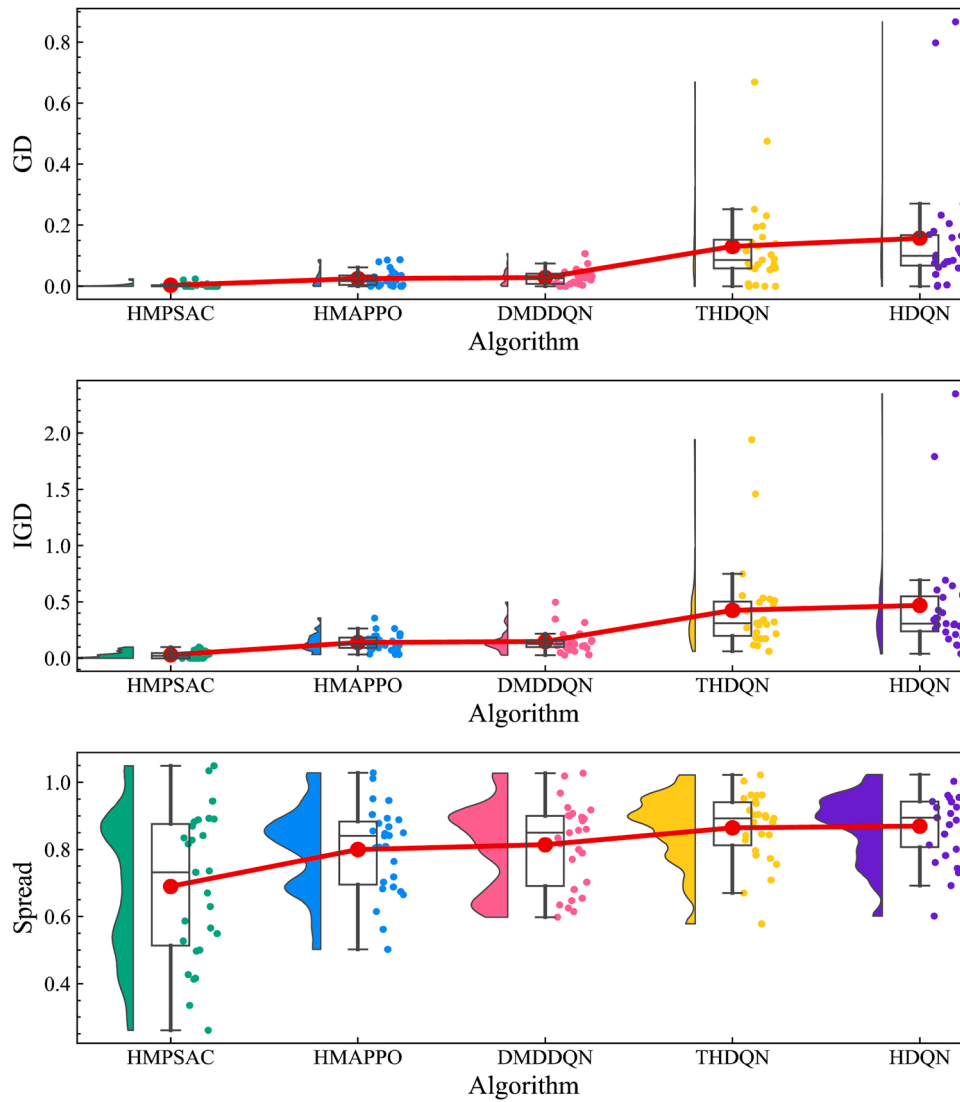


Fig. 11. The raincloud plot of three metrics (GD, IGD, and Spread).

signals but also the entropy (or uncertainty) of the policy. This characteristic aids in the comprehensive exploration of combinations of multiple objectives, preventing the algorithm from becoming trapped in local optima and enabling it to better adapt to multi-objective problems, generating a broader range of potential Pareto optimal solutions. (3) Hierarchical Multi-Strategy Network Structure: the HMPSAC algorithm adopts a hierarchical multi-strategy network structure, decomposing the strategy selection process into two tiers: the upper-level control strategy network and the lower-level objective strategy network. This structure allows the algorithm to select an appropriate objective strategy network at each decision point based on different targets, thereby effectively balancing multiple objectives.

To examine the statistical superiority of the HMPSAC algorithm over the comparison methods, we further conducted independent pairwise comparisons between the HMPSAC and reference methods using the Wilcoxon signed-rank test. Table 13 summarizes the results of the Wilcoxon test, performed by comparing each pair of methods. It shows the obtained ranks and the corresponding p values for each comparison. It can be observed from Table 13 that all p values are below the significance level $\alpha = 0.05$, indicating that the HMPSAC algorithm is superior to the comparison methods. Therefore, we can conclude that under the significance level $\alpha = 0.05$, the HMPSAC algorithm significantly outperforms the HMAPPO, DMDDQN, THDQN, and HDQN deep

reinforcement learning methods.

5.5. Comparison with metaheuristic algorithms

To verify the performance of the proposed method, this section uses instances from the public dataset for the FJSP with multiplicity characteristics, as described by Novas [63]. The public dataset instances focus on minimizing the makespan objective. The HMPSAC_C algorithm is a variant of the HMPSAC, specifically designed to optimize the makespan objective. Therefore, the HMPSAC_C is particularly well-suited for addressing this public dataset. The HMPSAC_C is compared with the GA algorithm (2020) [64] and the guided shuffled frog-leaping algorithm (GSFLA) (2023) [65]. These algorithms represent advanced methods recently developed for solving the FJSP with multiplicity characteristics. The evaluation metrics used include the average value C_a and the CPU computation time T_{cpu} . For each instance, the HMPSAC_C is run 30 times independently, and the average objective value, the best value, and the average computation time are recorded. The results of the two comparative algorithms are taken from Yang, et al. [65]. The comparison results are shown in Table 14. Instances P1–12 are classified as small-scale, P13–18 as medium-scale, and P19–24 as large-scale, based on the $R \times M$ scale.

The experimental results show that for small instances, GSFLA

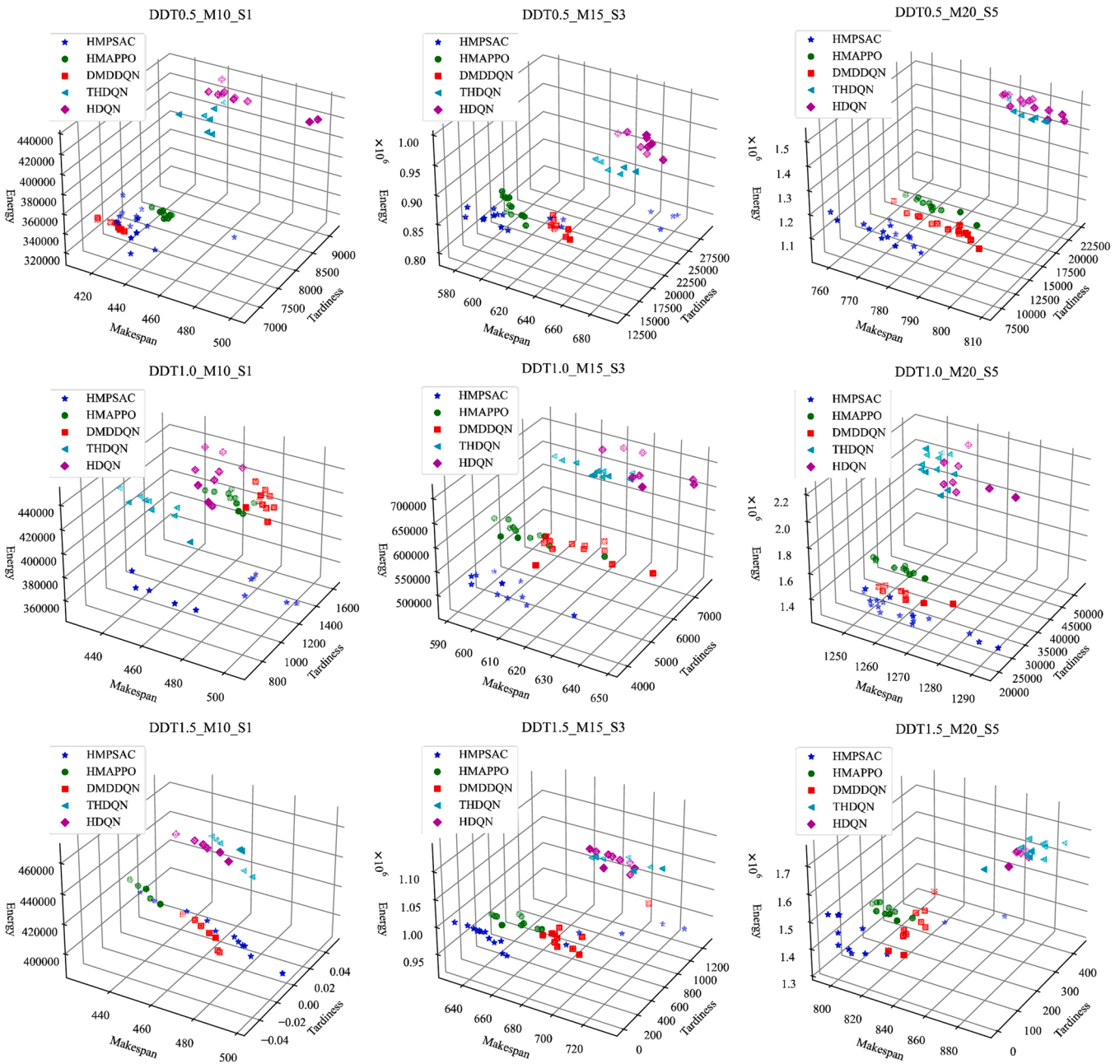


Fig. 12. Pareto fronts of HMPsAC and other four algorithms.

performs commendably, outperforming HMPsAC_C and GA in average solution quality. This is because GSFLA has a strong global search capability through multiple optimization iterations. Therefore, GSFLA can more easily find the optimal solution for small problems with small solution space. Although the HMPsAC_C algorithm has a high decision-making efficiency, its stochastic exploration characteristics during training and the quality of the scheduling rules chosen during decision-making make it difficult to obtain the global optimal solution even for small-scale problems. However, for medium and large instances, HMPsAC_C not only achieves superior average values compared to other algorithms but also obtains the best-known solutions for most instances. This is because the medium and large instances have a larger solution space than the small instances, which makes it difficult for the GSFLA and the GA to obtain the approximate optimal solution or the optimal solution. In terms of runtime efficiency, HMPsAC_C provides superior

solutions in 1 second for small instances. Furthermore, for medium and large instances, the computation time of other algorithms exceeds that of HMPsAC_C by more than 300 times, highlighting the exceptional computational efficiency of HMPsAC_C.

6. Conclusion

This paper presents a novel approach for the multi-objective multiplicity dynamic flexible job-shop scheduling problem with machine failures and new order arrivals, aiming to optimize the maximum completion time, total tardiness, and total energy consumption. A data-driven hierarchical multi-policy network framework is established, comprising a control policy network and three objective policy networks. To balance multiple objectives and obtain high-quality Pareto solutions, the proposed framework selects an objective policy network at

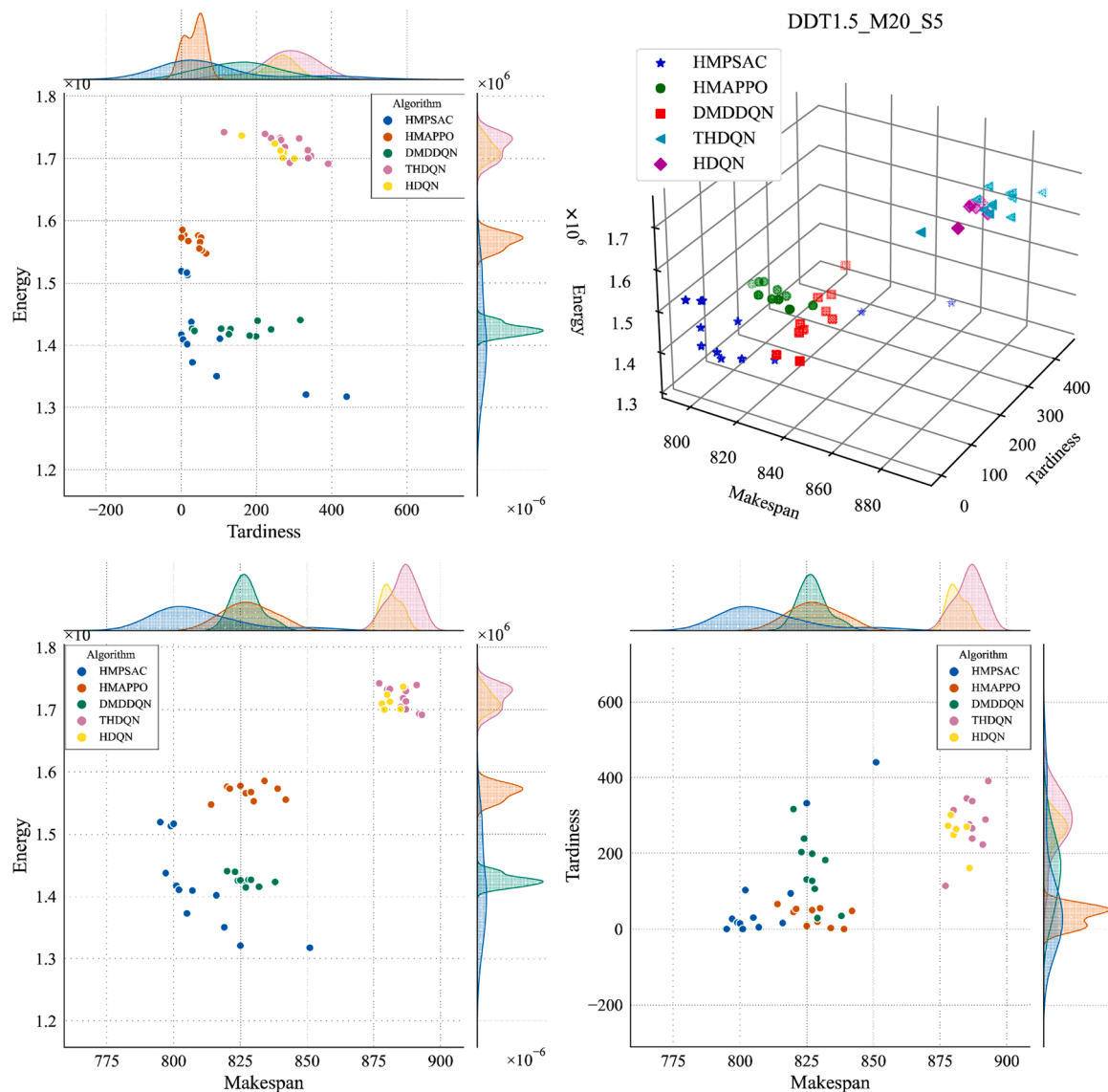


Fig. 13. The marginal combination plots for different dimensions.

decision points based on temporary targets identified by the control network. To further enhance this framework, state features, operation selection rules, and machine assignment rules are developed by integrating the fluid model with objective characteristics. Furthermore, a composite reward function is crafted to steer the algorithm towards the global Pareto front. The state feature and scheduling rule formulated in this research improves conventional static rule-based dispatching

Table 13
Results of the Wilcoxon test.

	HMPSAC vs.	R^-	R^+	p – value
GD	HMAPPO	101	301	0.00046894
	DMDDQN	114	285	0.00018884
	THDQN	54	345	0.00001982
	HDQN	21	357	0.00001040
IGD	HMAPPO	19	359	0.00001162
	DMDDQN	19	359	0.00000744
	THDQN	19	359	0.00000665
	HDQN	19	359	0.00000665
Spread	HMAPPO	153	225	0.01125638
	DMDDQN	105	273	0.00095507
	THDQN	91	287	0.00016997
	HDQN	91	287	0.00018705

methodologies by combining real time processing states under the workshop environment with dynamic parameters derived from fluid models. This novel approach allows for real-time tracking of dynamic events, enabling quick resolution in fluid model scenarios. Integrating these dynamic parameters with the optimization objectives refines the customization of state features and scheduling rules. This innovative strategy significantly improves the analytical and decision-making performance of DRL when applied to workshop scheduling scenarios. In addition, a two-stage training methodology incorporating maximum entropy reinforcement learning is proposed to improve training stability and enrich the diversity of Pareto solutions. The effectiveness of our approach has been validated through comparisons with well-known dispatching rules and other multi-objective dynamic scheduling algorithms. The results demonstrate the capability of our method to make efficient real-time decisions and generate a high-quality Pareto front. Our research provides a robust framework for addressing the complexities of modern manufacturing systems. In the future, we can integrate the fluid model and other network models (such as graph neural network and large language model) to address more complex constraints in the manufacturing systems.

Table 14
The comparison results of DDQN and other methods.

Instance	$R \times M$	GA		GSFLA		HMPSAC_C	
		C_a	T_{cpu}	C_a	T_{cpu}	C_a	T_{cpu}
P1	3 × 5	6570.2	27.3	6066.1	19.9	6226.5	0.33
P2	3 × 5	8967.6	32.6	8375.7	22.4	8542.1	0.42
P3	3 × 5	6448.9	25.4	5797.9	19.2	6701.5	0.28
P4	3 × 4	4008.7	24.6	3776.1	18.1	3832.1	0.23
P5	3 × 4	4724.6	23.5	4241.2	17.5	4306.8	0.26
P6	3 × 4	4483.1	26.2	4028.4	17.9	4066.5	0.25
P7	3 × 3	6074.5	24.9	5794.5	22.7	6212.7	0.38
P8	3 × 3	6172.5	21.6	5635.8	20.2	6050.9	0.32
P9	3 × 3	5764.3	26.1	5353	19.9	5646.3	0.30
P10	4 × 5	6608.2	38.8	5979.1	20.4	5999.2	0.21
P11	4 × 5	12252.3	43.1	11628.3	25.2	12187.5	0.37
P12	4 × 5	13410.3	50.8	12433.9	26.7	13084.1	0.45
P13	8 × 7	23749.8	622.6	21442.7	583.9	20591.2	1.80
P14	8 × 7	25890.4	641.5	23754.1	578.2	23011.6	1.82
P15	8 × 7	24336.6	638.9	22705.4	601.5	21692.9	1.91
P16	9 × 8	23986.3	836.1	23015.5	687.7	20632.8	2.50
P17	9 × 8	24667.3	824.5	23342.1	693.1	21522.7	2.45
P18	9 × 8	23898	762.3	22690.5	675.5	21757.6	2.41
P19	11 × 8	26755.1	1737.6	25021.8	1480.3	23070.6	2.81
P20	11 × 8	25988.1	1785.1	24501.4	1659.2	22710.6	3.00
P21	11 × 8	27942	1665.8	26012.3	1761.2	24497.0	3.21
P22	12 × 8	26788.1	2001.3	25820.2	1829.9	24477.4	3.32
P23	12 × 8	37092.1	2277.6	35628.8	1920.5	33024.7	4.53
P24	12 × 8	27279.6	2176.5	25790.8	1643.2	25351.5	

CRedit authorship contribution statement

Ding Linshan: Writing – original draft, Methodology, Data curation.
Luo Dan: Visualization, Investigation, Formal analysis, Writing – review & editing.
Guan Zailin: Supervision, Project administration, Conceptualization.
Yue Lei: Writing – review & editing, Visualization, Project administration.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the Guangzhou Basic and Applied Basic Research Project (No. 2025A03J0154), the Science and Technology Planning Project of Guangzhou City (No. 2023A03J0120).

References

[1] Elyasi M, Altan B, Ekici A, Özener OÖ, Yanıkoğlu İ, Dolgui A. Production planning with flexible manufacturing systems under demand uncertainty. *Int J Prod Res* 2024;62:157–70.

[2] Baykasoğlu A, Madenoğlu FS, Hamzadayı A. Greedy randomized adaptive search for dynamic flexible job-shop scheduling. *J Manuf Syst* 2020;56:425–51.

[3] Tian Z, Jiang X, Liu W, Li Z. Dynamic energy-efficient scheduling of multi-variety and small batch flexible job-shop: a case study for the aerospace industry. *Comput Ind Eng* 2023;178:109111.

[4] Monch L, Fowler JW, Dauzere-Peres S, Mason SJ, Rose O. A survey of problems, solution techniques, and future challenges in scheduling semiconductor manufacturing operations. *J Sched* 2011;14:583–99.

[5] Atzori L, Iera A, Morabito G. The Internet of things: a survey. *Comput Netw* 2010;54:2787–805.

[6] Sadiku MNO, Musa SM, Momoh OD. Cloud computing: opportunities and challenges. *IEEE Potentials* 2014;33:34–6.

[7] Gu W, Li Y, Tang D, Wang X, Yuan M. Using real-time manufacturing data to schedule a smart factory via reinforcement learning. *Comput Ind Eng* 2022;171:108406.

[8] Jiang B, Ma Y, Chen L, Huang B, Huang Y, Guan L. A review on intelligent scheduling and optimization for flexible job shop. *Int J Control, Autom Syst* 2023;21:3127–50.

[9] Gholami M, Zandieh M. Integrating simulation and genetic algorithm to schedule a dynamic flexible job shop. *J Intell Manuf* 2009;20:481–98.

[10] Zhenyu G, Linfeng L, Jiajia Z, Guorong L. A Flexible Job-shop Rescheduling Method By Considering The Machine Equipment Availability. in: 2016 Chinese Control and Decision Conference (CCDC). IEEE; 2016. p. 4898–902.

[11] Yang Y, Huang M, Wang ZY, Zhu QB. Robust scheduling based on extreme learning machine for bi-objective flexible job-shop problems with machine breakdowns. *Expert Syst Appl* 2020;158:113–545.

[12] Nouri M, Bekrar A, Trentesaux D. Towards energy efficient scheduling and rescheduling for dynamic flexible job shop problem. *Ifac-Pap* 2018;51:1275–80.

[13] Wei L, He J, Guo Z, Hu Z. A multi-objective migrating birds optimization algorithm based on game theory for dynamic flexible job shop scheduling problem. *Expert Syst Appl* 2023;227:120–268.

[14] Buddala R, Mahapatra SS. Two-stage teaching-learning-based optimization method for flexible job-shop scheduling under machine breakdown. *Int J Adv Manuf Technol* 2019;100:1419–32.

[15] Teymourifar A, Ozturk G, Ozturk ZK, Bahadır O. Extracting new dispatching rules for multi-objective dynamic flexible job shop scheduling with limited buffer spaces. *Cogn Comput* 2020;12:195–205.

[16] Ghaleb M, Taghipour S, Zolfaghariania H. Real-time integrated production-scheduling and maintenance-planning in a flexible job shop with machine deterioration and condition-based maintenance. *J Manuf Syst* 2021;61:423–49.

[17] Lv Y, Li C, Tang Y, Kou Y. Toward energy-efficient rescheduling decision mechanisms for flexible job shop with dynamic events and alternative process plans. *Ieee T Autom Sci Eng* 2021;19:3259–75.

[18] Li C, Zheng P, Yin Y, Wang B, Wang L. Deep reinforcement learning in smart manufacturing: a review and prospects. *CIRP J Manuf Sci Technol* 2023;40:75–101.

[19] T.P. Lillicrap, J.J. Hunt, A. Pritzel, N. Heess, T. Erez, Y. Tassa, D. Silver, D. Wierstra, Continuous control with deep reinforcement learning, arXiv preprint arXiv:1509.02971, (2015).

[20] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, arXiv preprint arXiv:1707.06347, (2017).

[21] Watkins CJCH, Dayan P. Q-learning. *Mach Learn* 1992;8:279–92.

[22] V. Mnih, K. Kavukcuoglu, D. Silver, A. Graves, I. Antonoglou, D. Wierstra, M. Riedmiller, Playing atari with deep reinforcement learning, arXiv preprint arXiv:1312.5602, (2013).

[23] Van Hasselt H, Guez A, Silver D. Deep reinforcement learning with double Q-learning. *Proc AAAI Conf Artif Intell* 2016. 2094–2011.

[24] Zhao M, Li X, Gao L, Wang L, Xiao M. An improved Q-learning based rescheduling method for flexible job-shops with machine failures. in: 2019 IEEE 15th International Conference on Automation Science and Engineering (CASE). Vancouver, BC, Canada: IEEE; 2019. p. 331–7.

[25] Baer S, Turner D, Mohanty P, Samsonov V, Bakakeu R, Meisen T. Multi agent deep q-network approach for online job shop scheduling in flexible manufacturing. : ICMSMM 2020: Int Conf Manuf Syst Mult Mach, Tokyo, Jpn 2020:78–86.

[26] Luo S. Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning. *Appl Soft Comput* 2020;91:106208.

[27] Palombinari JA, Martínez EC. End-to-end on-line rescheduling from Gantt chart images using deep reinforcement learning. *Int J Prod Res* 2021;60:4434–63.

[28] Yang S, Xu Z. Intelligent scheduling and reconfiguration via deep reinforcement learning in smart manufacturing. *Int J Prod Res* 2021;60:4936–53.

[29] Chang J, Yu D, Hu Y, He W, Yu H. Deep reinforcement learning for dynamic flexible job shop scheduling with random job arrival. *Processes* 2022;10:760–80.

- [30] Liu R, Piplani R, Toro C. Deep reinforcement learning for dynamic scheduling of a flexible job shop. *Int J Prod Res* 2022;60:4049–69.
- [31] Gui Y, Tang D, Zhu H, Zhang Y, Zhang Z. Dynamic scheduling for flexible job shop using a deep reinforcement learning approach. *Comput Ind Eng* 2023;180:109255.
- [32] Zhang L, Feng Y, Xiao Q, Xu Y, Li D, Yang D, Yang Z. Deep reinforcement learning for dynamic flexible job shop scheduling problem considering variable processing times. *J Manuf Syst* 2023;71:257–73.
- [33] Lang S, Behrendt F, Lanzerath N, Reggelin T, Muller M. Integration of deep reinforcement learning and discrete-event simulation for real-time scheduling of a flexible job shop production. in: 2020 Winter Simulation Conference (WSC). Orlando, FL, USA: IEEE; 2020. p. 3057–68.
- [34] Luo S, Zhang L, Fan Y. Real-time scheduling for dynamic partial-no-wait multiobjective flexible job shop by deep reinforcement learning. *Ieee T Autom Sci Eng* 2021;19:3020–38.
- [35] Luo S, Zhang L, Fan Y. Dynamic multi-objective scheduling for flexible job shop by deep reinforcement learning. *Comput Ind Eng* 2021;159:107489.
- [36] Li Y, Gu W, Yuan M, Tang Y. Real-time data-driven dynamic scheduling for flexible job shop with insufficient transportation resources using hybrid deep Q network. *Robot Cim-Int Manuf* 2022;74:102283.
- [37] Wang H, Cheng J, Liu C, Zhang Y, Hu S, Chen L. Multi-objective reinforcement learning framework for dynamic flexible job shop scheduling problem with uncertain events. *Appl Soft Comput* 2022;131:109717.
- [38] Wu Z, Fan H, Sun Y, Peng M. Efficient multi-objective optimization on dynamic flexible job shop scheduling using deep reinforcement learning approach. *Processes* 2018;11(2023).
- [39] Zhang J, Guo B, Ding X, Hu D, Tang J, Du K, Tang C, Jiang Y. An adaptive multi-objective multi-task scheduling method by hierarchical deep reinforcement learning. *Appl Soft Comput* 2024;154:111342.
- [40] Kulkarni TD, Narasimhan K, Saeedi A, Tenenbaum J. Hierarchical deep reinforcement learning: Integrating temporal abstraction and intrinsic motivation. *Adv Neural Inf Process Syst* 2016;29.
- [41] Pateria S, Subagdja B, Tan AH, Quek CJACS. Hierarchical reinforcement learning: a comprehensive survey. *ACM Comput Surv* 2021;54:1–35.
- [42] Bertsimas D, Gamarnik D. Asymptotically optimal algorithms for job shop scheduling and packet routing. *J Algorithms* 1999;33:296–318.
- [43] Boudoukh T, Penn M, Weiss G. Scheduling job shops with some identical or similar jobs. *J Sched* 2001;4:177–99.
- [44] Bertsimas D, Sethuraman J. From fluid relaxations to practical algorithms for job shop scheduling: the makespan objective. *Math Program* 2002;92:61–102.
- [45] Dai JG, Weiss G. A fluid heuristic for minimizing makespan in job shops. *Oper Res* 2002;50:692–707.
- [46] Bertsimas D, Gamarnik D, Sethuraman J. From fluid relaxations to practical algorithms for high-multiplicity job-shop scheduling: the holding cost objective. *Oper Res* 2003;51:798–813.
- [47] Nazarathy Y, Weiss G. A fluid approach to large volume job shop scheduling. *J Sched* 2010;13:509–29.
- [48] Masin M, Raviv T. Linear programming-based algorithms for the minimum makespan high multiplicity jobshop problem. *J Sched* 2014;17:321–38.
- [49] Gu MZ, Lu XW, Gu JW. An asymptotically optimal algorithm for large-scale mixed job shop scheduling to minimize the makespan. *J Comb Optim* 2017;33:473–95.
- [50] Gu JW, Gu MZ, Lu XW, Zhang Y. Asymptotically optimal policy for stochastic job shop scheduling problem to minimize makespan. *J Comb Optim* 2018;36:142–61.
- [51] Shen XN, Yao X. Mathematical modeling and multi-objective evolutionary algorithms applied to dynamic flexible job shop scheduling problems. *Inf Sci* 2015; 298:198–224.
- [52] Vinod V, Sridharan R. Dynamic job-shop scheduling with sequence-dependent setup times: simulation modeling and analysis. *Int J Adv Manuf Tech* 2008;36: 355–72.
- [53] Haarnoja T, Zhou A, Abbeel P, Levine S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. in: International conference on machine learning. PMLR; 2018. p. 1861–70.
- [54] Holthaus O. Scheduling in job shops with machine breakdowns: an experimental study. *Comput Ind Eng* 1999;36:137–62.
- [55] Zitzler E, Thiele L. Multiobjective evolutionary algorithms: a comparative case study and the strength Pareto approach. *IEEE Trans Evolut Comput* 1999;3: 257–71.
- [56] Rajendran C, Holthaus OJEjoor. A comparative study of dispatching rules in dynamic flowshops and jobshops. *Eur J Oper Res* 1999;116:156–70.
- [57] Holthaus O, Rajendran CJPP. Control, Efficient job shop dispatching rules: further developments. *Prod Plan Control* 2000;11:171–8.
- [58] Wei Y, Zhao M. Composite rules selection using reinforcement learning for dynamic job-shop scheduling. in: IEEE Conference on Robotics, Automation and Mechatronics, 2004. IEEE; 2004. p. 1083–8.
- [59] Dominic PD, Kaliyamoorthy S, Kumar MS. Efficient dispatching rules for dynamic job shop scheduling. *Int J Adv Manuf Technol* 2004;24:70–5.
- [60] Chen B, Matis TI. A flexible dispatching rule for minimizing tardiness in job shop scheduling. *Int J Prod Econ* 2013;141:360–5.
- [61] Zitzler E, Deb K, Thiele L. Comparison of multiobjective evolutionary algorithms: empirical results. *Evol Comput* 2000;8:173–95.
- [62] Deb K, Pratap A, Agarwal S, Meyarivan T. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Trans Evolut Comput* 2002;6:182–97.
- [63] Novas JM. Production scheduling and lot streaming at flexible job-shops environments using constraint programming. *Comput Ind Eng* 2019;136:252–64.
- [64] Yan G, Aimin W, Zijin Z. Flexible job-shop scheduling with setups and variable sublots. in: In 2020 IEEE 11th International Conference on Mechanical and Intelligent Manufacturing Technologies (ICMIMT) IEEE, Cape town, South Africa. 2020. p. 187–92.
- [65] Yang Y, Song Y, Guo W, Lei Q, Sun A, Fan L. Guided shuffled frog-leaping algorithm for flexible job shop scheduling problem with variable sublots and overlapping in operations. *Comput Ind Eng* 2023;180:109209.